PACKTRACE – GPS-BASED GROUP RIDE TRACKING SYSTEM

A Web-Based Real-Time Location Sharing Application for Bike Riders

A MINI PROJECT REPORT

Submitted by

ALAN JOHNY
Reg. No: MLM25MCA2015

To

the APJ Abdul Kalam Technological University

in Partial fulfillment of the requirements for

the award of the degree

of

MASTER OF COMPUTER APPLICATIONS

Under the Guidance of

Ms. DIVYA S B
(HOD&Assistant Professor, Department of Computer Applications)

DEPARTMENT OF COMPUTER APPLICATIONS
MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR
(Affiliated to APJ Abdul Kalam Technological University)

OCTOBER 2026

MANGALAM COLLEGE OF ENGINEERING

Accredited by NAAC& ISO 9001:2000 Certified Institution

DEPARTMENT OF COMPUTER APPLICATIONS

VISION

To become a centre of excellence in computer applications,competent in the global ecosystem with technical knowledge,innovation with a sense of social commitment.

MISSION

- To serve with state of the art education,foster advanced research and cultivate innovation in the field of computer applications.
- To prepare learners with knowledge skills and critical thinking to excel in the technological landscape and contribute positively to society.

Program Educational Objectives

- PEO I :Graduates will possess a solid foundation and in-depth understanding of computer applications and will be equipped to analyze real-world problems, design and create innovative solutions, and effectively manage and maintain these solutions in their professional careers.
- PEO II: Graduates will acquire technological advancements through continued education, lifelong learning and research, thereby making meaningful contributions to the field of computing.
- PEO III: Graduates will cultivate team spirit, leadership, communication skills, ethics, and social values, enabling them to apply their understanding of the societal impacts of computer applications effectively.

Program Specific Outcomes

- **PSO I**: Apply advanced technologies through innovations to enhance the efficiency of design and development.
- **PSO II**: Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges.

MAPPING OF PO-PSO-SDG 1.

MAPPING WITH PROGRAM OUTCOMES (POs): -

SL. NO	POs ADDRESSED	RELEVANCE TO PROJECT
1	PO1	Applied Python programming, Django web development, database design and geographic distance computation to design and build the group ride tracking application.
2	PO2	Analysed the difficulty of keeping the riders of a moving group in view and the limitations of general messaging applications, and identified the need for a dedicated group ride tracking system.
3	PO3	Designed and developed PackTrace to share rider locations in real time on a common map, with disconnect handling, route deviation detection, SOS alerts and ride history, contributing to rider safety (SDG 3, SDG 11).
4	PO4	Used modern tools such as Django, Django Channels (WebSocket), the browser Geolocation API, the Google Maps Platform, StarUML and GitHub, with an understanding of the limitations of GPS accuracy.
5	PO5	Carried out the project individually under the guidance of the project guide, with periodic reviews of progress.
6	PO6	Planned the work as modular stages — authentication, ride management, live tracking, alerts, history and administration — using version control and free and open-source tools.
8	PO8	Learned real-time web technologies such as WebSockets and web mapping beyond the syllabus.

LIST OF PROGRAM OUTCOMES (POs):

PO1 (Foundation Knowledge): Apply knowledge of mathematics, programming logic and coding fundamentals for solution architecture and problem solving.

PO2 (Problem Analysis): Identify, review, formulate and analyze problems for primarily focusing on customer requirements using critical thinking frameworks.

PO3 (Development of Solutions): Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.

PO4 (Modern Tool Usage): Select, adapt and apply modern computational tools such as development of algorithms with an understanding of the limitations including human biases.

PO5 (Individual and Teamwork): Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.

PO6 (Project Management and Finance): Use the principles of project management such as scheduling, work breakdown structure and be conversant with the principles of Finance for profitable project management.

PO7 (Ethics): Commit to professional ethics in managing software projects with financial aspects. Learn to use new technologies for cyber security and insulate customers from malware.

PO8 (Life-long learning): Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

2. MAPPING WITH PROGRAM SPECIFIC OUTCOMES (PSOs):

SL. NO	PSOs ADDRESSED	RELEVANCE TO PROJECT
1	PSO 1	Applied modern technologies such as Django, WebSocket communication, browser geolocation and web mapping to design and develop the group ride tracking system.
2	PSO 2	Used computing principles such as geographic distance computation, realtime data distribution and structured database design to develop a practical solution for safer group travel.

LIST OF PROGRAM SPECIFIC OUTCOMES (PSOs):

PSO 1: Apply advanced technologies through innovations to enhance the efficiency of design development.

PSO 2: Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges.

3. MAPPING WITH SUSTAINABLE DEVELOPMENT GOALS (SDGs):

SDG NO	SDGs ADDRESSED	RELEVANCE TO PROJECT
SDG 8	Decent work and economic growth	PackTrace can support safer and more efficient road travel for people who use motorcycles for work, commuting, delivery, or other occupational purposes. Real-time group tracking, route monitoring, and SOS features can help improve coordination and reduce risks during work-related travel. The contribution is indirect, as PackTrace does not directly create employment or economic growth.
SDG 9	Industry, Innovation and Infrastructure	The project applies real-time web, geolocation and web mapping technologies to build a coordination tool for group travel.
SDG 11	Sustainable Cities and Communities	Group travel on roads is made safer and better coordinated, and riding communities can organise rides with a shared view of the group.

SUSTAINABLE DEVELOPMENT GOALS (SDGs):

SDG 1 – No Poverty: End poverty in all its forms everywhere.

SDG 2 – Zero Hunger: End hunger, achieve food security and improved nutrition, and promote sustainable agriculture.

SDG 3 – Good Health and Well-Being: Ensure healthy lives and promote well-being for all at all ages.

SDG 4 – Quality Education: Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.

SDG 5 – Gender Equality: Achieve gender equality and empower all women and girls.

SDG 6 – Clean Water and Sanitation: Ensure availability and sustainable management of water and sanitation for all.

SDG 7 – Affordable and Clean Energy: Ensure access to affordable, reliable, sustainable, and modern energy for all.

SDG 8 – Decent Work and Economic Growth: Promote sustained, inclusive, and sustainable economic growth, full and productive employment, and decent work for all.

SDG 9 – Industry, Innovation, and Infrastructure: Build resilient infrastructure, promote inclusive and sustainable industrialization, and foster innovation.

SDG 10 – Reduced Inequality: Reduce inequality within and among countries.

SDG 11 – Sustainable Cities and Communities: Make cities and human settlements inclusive, safe, resilient, and sustainable.

SDG 12 – Responsible Consumption and Production: Ensure sustainable consumption and production patterns.

SDG 13 – Climate Action: Take urgent action to combat climate change and its impacts.

SDG 14 – Life Below Water: Conserve and sustainably use the oceans, seas, and marine resources.

SDG 15 – Life on Land: Protect, restore, and promote sustainable use of terrestrial ecosystems, manage forests sustainably, combat desertification, halt and reverse land degradation, and halt biodiversity loss.

SDG 16 – Peace, Justice, and Strong Institutions: Promote peaceful and inclusive societies, provide access to justice for all, and build effective, accountable, and inclusive institutions.

SDG 17 – Partnerships for the Goals: Strengthen the means of implementation and revitalize the global partnership for sustainable development.

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR
DEPARTMENT OF COMPUTER APPLICATIONS
OCTOBER 2026



DECLARATION

I hereby certify that the work which is being presented in the project entitled "PACKTRACE – GPS-BASED GROUP RIDE TRACKING SYSTEM" submitted in the DEPARTMENT OF COMPUTER APPLICATIONS is an authentic record of my own work carried under the supervision of Ms. Divya S B, HOD & Assistant professor. This study has not been submitted to any other institution or university for the award of any other degree. This report has been checked for plagiarism by the college and the similarity index is within permissible limits set by the college.

Date:
Place:

Name & Signature of Student
ALAN JOHNY

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR

DEPARTMENT OF COMPUTER APPLICATIONS

OCTOBER 2026



CERTIFICATE

Certified that the report entitled "Packtrace-GPS BASED GROUP RIDE TRACKING SYSTEM" submitted by ALAN JOHNY (MLM25MCA-2015) to the APJ Abdul Kalam Technological University in partial fulfilment of the requirements for the award of the Degree of Master of Computer Applications is a bonafide record of the MINI PROJECT work carried out by the student under our guidance and supervision during the academic year 2026-27.

PROJECT GUIDE	***PROJECT COORDINATOR***
Ms. Divya S B	**Ms. Athira R Kurup**
HOD & Assistant professor	**Assistant Professor**
Department of Computer Applications	**Department of Computer Applications**

HEAD OF THE DEPARTMENT	***INTERNAL EXAMINER***
Ms. Divya S B	
HOD & Assistant professor	
Department of Computer Applications	

ACKNOWLEDGEMENT

I am greatly indebted to the authorities of Mangalam College of Engineering for providing the necessary facilities to successfully complete my Project on the topic "PACKTRACE – GPS-BASED GROUP RIDE TRACKING SYSTEM".

I express my sincere thanks to Dr. Vinodh P Vijayan, Principal, Mangalam College of Engineering for providing the facilities to complete my Project successfully.

I thank and express my sincere gratitude to Ms. Divya S B, HOD & Assistant Professor, Department of Computer Applications, Mangalam College of Engineering, for her invaluable help and support, which helped me in successfully completing this Project work.

I express my gratitude to my Internal Guide, Ms. Divya S B, HOD & Assistant Professor, Department of Computer Applications, for the suggestions and encouragement which helped in the successful completion of my Project.

Furthermore, I would like to acknowledge with much appreciation the crucial role of the faculty, especially the Project Coordinator, Ms. Athira R Kurup, Department of Computer Applications, Mangalam College of Engineering, who gave the permission to use all the required equipment and the necessary resources to complete the presentation and report.

Finally, I would like to express my heartfelt thanks to my parents, who were very supportive both financially and mentally, and for their encouragement to achieve my goal.

ALAN JOHNY
(MLM25MCA2015)

ABSTRACT

Keeping every rider of a moving group in view is difficult on long rides. Riders become separated by traffic, speed differences and stops, and the tools they normally use — messaging applications with individual live-location sharing, navigation applications and phone calls — do not give the group a single, shared picture of where everyone is.

PackTrace is a web-based real-time location sharing application developed to address this problem. A rider registers, logs in, and creates a ride group or joins one created by another rider. During a ride, the rider's browser obtains the device's GPS position through the Geolocation API and sends it to a Python/Django backend, which communicates with the other members of the group in real time using WebSocket communication. The positions of all riders are shown on a common map built with the Google Maps Platform (Maps JavaScript API), so that every member can see the others as markers, together with ride information on a centralised dashboard.

In addition to live tracking, PackTrace provides detection of a rider who stops sending updates, in which case the last known location is retained, detection of deviation from the planned route, an SOS emergency alert that notifies the group, a ride history of completed rides, and an administration interface for managing the system. The application is developed using Python 3.10 and Django 5.2 with HTML, CSS and JavaScript on the client side.

This report presents the requirements, literature review, system diagrams, methodology, implementation and testing of PackTrace. It also discusses the system's limitations and possible future enhancements. PackTrace demonstrates how a browser-based platform can provide a shared view of group members and improve awareness when riders become separated.

Keywords: GPS tracking, real-time location sharing, group ride tracking, Django, WebSocket, Google Maps Platform, geolocation, route deviation detection, SOS alert, ride history.

Mapping with Sustainable Development Goals	**Goals Addressed**
SDGs to which this project contributes	SDG 8 Decent Work and Economic Growth SDG 9 – Industry, Innovation and Infrastructure SDG 11 – Sustainable Cities and Communities

TABLE OF CONTENTS

ACKNOWLEDGEMENT **I**

ABSTRACT **II**

TABLE OF CONTENTS **III**

LIST OF FIGURES **V**

LIST OF TABLES **VI**

LIST OF ABBREVIATIONS **VII**

Chapter 1 INTRODUCTION **1**

1.1 Background 1

1.2 Motivation 1

1.3 Scope and Objectives 2

1.4 Problem Statement 3

Chapter 2 REQUIREMENT AND ANALYSIS **4**

2.1 Existing System and Problem Statement 4

2.2 Solution Overview 4

2.3 Feasibility Study 5

2.4 Requirement Specification 5

Chapter 3 LITERATURE REVIEW **9**

3.1 GNN-RMNet: Graph Neural Networks and GPS Analytics for Driver Behavior and Route Optimization in Logistics 9

3.2 Real-Time GPS Tracking System for IoT-Enabled Connected Vehicles 9

3.3 Towards a Sustainable City for Cyclists: Promoting Safety through a Mobile Sensing Application 10

3.4 Communicating and Displaying Real-Time Data with WebSocket 10

3.5 Accuracy of Phone-Based GPS Positioning 11

3.6 WreckWatch: Automatic Traffic Accident Detection and Notification with Smartphones 12

3.7 Location Disclosure to Social Relations: Why, When, & What People Want to Share 12

3.8 Location Privacy in Pervasive Computing 13

3.9 Virtues of the Haversine 13

3.10 Summary of the Literature 14

Chapter 4 DIAGRAMS **15**

4.1 Data Flow Diagram 17

4.2 Use Case Diagram 18
4.3 Class Diagram 19
4.4 State Diagram 20
4.5 Activity Diagram 21
4.6 Sequence Diagram 21
4.7 Entity Relationship Diagram 22
4.8 Database Design 23
Chapter 5 METHODOLOGY 26
5.1 Module Descriptions 26
5.2 Algorithms 34
5.3 Architecture 35
5.4 Case Study 36
Chapter 6 IMPLEMENTATION DETAILS 38
6.1 Implementation Procedures 38
6.2 Testing Methods and Results 39
Chapter 7 ADVANTAGES AND DISADVANTAGES 45
7.1 Advantages 45
7.2 Disadvantages 45
Chapter 8 RESULT AND DISCUSSION 46
8.1 Analysis 46
Chapter 9 CONCLUSION 48
9.1 Summary 48
9.2 Limitations 48
9.3 Future Scope 49
9.4 Applications 49
Chapter 10 APPENDICES 50
10.1 Sample code 50
Chapter 11 REFERENCES 52

LIST OF FIGURES

4.1	DFD Level 0	17
4.2	DFD Level 1	18
4.3	Use Case Diagram	19
4.4	Class Diagram (design-stage conceptual model)	20
4.5	State Diagram	21
4.6	Activity Diagram	21
4.7	Sequence Diagram	22
4.8	Entity Relationship Diagram	23
5.1	Web Application Wireframes	27
5.2	Register	27
5.3	Login	28
5.4	Create / Join ride screen	28
5.5	Dashboard layout	29
5.6	Live Google Maps view	30
5.7	SOS alert and notification screen	33
5.8	Ride history screen	33
5.9	Administration interface	34
5.10	System Architecture of PackTrace	36
6.1	Evidence of user acceptance testing	44

LIST OF TABLES

2.1	Functional requirements of PackTrace	6
2.2	Non-functional requirements	6
2.3	Hardware requirements	7
2.4	Software requirements	7
3.1	Literature review of GPS-based tracking and related systems	14
4.1	USER table (application user)	23
4.2	RIDE_GROUP table	24
4.3	RIDE_MEMBER table	24
4.4	LOCATION_LOG table	24
4.5	SOS_ALERT table	24
4.6	RIDE_HISTORY table	25
4.7	NOTIFICATION table	25
5.1	Modules of PackTrace and available evidence	26
5.2	Map implementation in the final application	31
6.1	Security measures	39
6.2	User acceptance test cases	40
8.1	Status of the objectives	47

LIST OF ABBREVIATIONS

ABBREVIATION		FULL FORM
PackTrace	-	GPS-Based Group Ride Tracking System
GPS	-	Global Positioning System
API	-	Application Programming Interface
UI	-	User Interface
HTML	-	HyperText Markup Language
CSS	-	Cascading Style Sheets
JS	-	JavaScript
HTTP	-	HyperText Transfer Protocol
HTTPS	-	HTTP Secure
WS	-	WebSocket
WSS	-	WebSocket Secure
REST	-	Representational State Transfer
JSON	-	JavaScript Object Notation
SOS	-	Emergency distress signal
DFD	-	Data Flow Diagram
UML	-	Unified Modeling Language
ER	-	Entity Relationship
ORM	-	Object Relational Mapper
SQL	-	Structured Query Language
DB	-	Database
CSRF	-	Cross-Site Request Forgery
Django	-	Django Web Framework
UAT	-	User Acceptance Testing
IDE	-	Integrated Development Environment
SDG	-	Sustainable Development Goal
PO / PSO	-	Programme Outcome / Programme Specific Outcome

CHAPTER 1

INTRODUCTION

1.1 Background

Riding together has become a popular leisure and touring activity. Groups of friends, riding clubs and touring communities regularly travel long distances on motorcycles and bicycles, often on highways and ghat roads where traffic conditions and speeds change continuously. On such routes the members of a group seldom stay together. Differences in speed, traffic signals, fuel stops and road conditions stretch the group out over several kilometres, and a rider who takes a wrong turn or stops for a breakdown may remain unnoticed until the group has moved far ahead.

Being separated from the group creates practical and safety problems. The group leader cannot confirm that every member is following the planned route, riders at the back cannot tell how far behind they are, and in an emergency the other riders may not know where the affected rider was last seen. Riders also have to avoid operating a mobile phone while riding, so coordination through calls and typed messages is both inefficient and unsafe.

Smartphones with built-in GPS receivers and mobile data connectivity make it possible to address these problems. If each rider's position is shared continuously and displayed on a map that every member of the group can see, the group acquires a common picture of where everybody is. Modern web technologies such as the browser Geolocation API, WebSocket communication and web mapping services allow this to be done inside an ordinary web browser, without installing a separate application.

PackTrace is developed on this basis. It is a web-based group ride tracking application in which riders register, create or join a ride group, and share their live location during a ride. The positions of all members are shown on a common map on a centralised dashboard. The system also detects riders who stop sending updates, detects deviation from the planned route, provides an SOS emergency alert and stores completed rides in a ride history. PackTrace is a web application accessed through a browser; it is not a native Android or iOS application.

1.2 Motivation

The motivation for developing PackTrace arises from the practical difficulty of coordinating a group of riders on the road. Riding groups currently depend on a mixture of messaging applications, individual navigation applications and phone calls. Each of these tools was designed for a different purpose: a messaging application shares one person's location inside a conversation, and a navigation application guides one user along a route.

Neither presents the whole group on a single map, and neither understands the idea of a ride with a leader, members and a planned route.

Another motivation is rider safety. When a rider drops out of the group because of a breakdown, a fall or a lost signal, the other riders may not notice for some time, and the last known position of that rider may not be known. A system that shows every rider continuously, marks a rider whose updates have stopped and offers a one-action SOS alert can reduce this delay.

A further motivation is the availability of suitable technology. The Geolocation API gives web pages access to the device position, WebSocket communication allows a server to push updates to many users immediately, and the Google Maps Platform provides interactive maps. Django provides a mature framework for authentication, data storage and administration. The motivation of this project is therefore to use these technologies to build a simple, centralised and practical ride-monitoring tool for riding groups.

1.3 Scope and Objectives

The scope of PackTrace is a web-based group ride tracking system for riders who travel together. Within this scope the system covers rider registration and login; creation of ride groups and joining them; continuous sharing of live location during a ride; display of all riders on an interactive map; detection of riders who stop sending updates; detection of deviation from the planned route; SOS emergency alerts; ride history; and an administration interface. The system runs in a web browser on a laptop or a smartphone with GPS. Native mobile applications, offline tracking, voice navigation and automatic accident detection are outside the scope of the present work and are listed as future scope.

The main objectives of PackTrace are:

- To provide real-time location sharing among the riders of a group.
- To allow users to create ride groups and to join them.
- To display multiple riders on a common map and on a centralised ride dashboard.
- To track the movement of riders during an active ride and show their route.
- To improve awareness when riders become separated, by detecting riders who stop sending updates and showing their last known location.
- To detect when a rider deviates from the planned route.
- To provide an SOS emergency alert that notifies the group.
- To maintain a ride history so that completed rides can be reviewed.
- To provide secure authentication and an administration interface.
- To provide a scalable web architecture based on Python and Django.

1.4 Problem Statement

During long group rides involving multiple riders, it can be difficult to continuously monitor the location of every member. Conventional messaging applications may provide individual live-location sharing but do not provide a dedicated centralised ride-monitoring environment designed around a riding group. Riders may become separated, lose awareness of other members, or have difficulty determining the current position of the group.

PackTrace addresses this by providing a web-based platform where riders can participate in a ride group and view the real-time locations of group members on a common map. The platform further supports ride-level safety functions — handling of disconnected riders, route deviation detection and SOS alerts — and keeps a history of completed rides.

CHAPTER 2

REQUIREMENT AND ANALYSIS

2.1 Existing System and Problem Statement

Riding groups at present coordinate through general-purpose tools. Messaging applications such as WhatsApp and Telegram allow a user to share a live location with a chat for a limited period. Navigation applications, including the location-sharing facility of Google Maps, guide a single user and can share that user's position with selected contacts. Riders also coordinate by phone calls, messages and, in some groups, helmet intercoms.

These tools work well for their original purposes, but they have limitations when used to manage a group ride. Each shared location is a separate item, so a rider has to open several shares to see where the others are; there is no single map that shows the whole group together. There is no idea of a ride with a group code, a leader and a planned route, and therefore no ride-specific history. There is no built-in way to detect that a rider has dropped off the map or has left the route, and no one-action emergency alert addressed to the whole group. Checking a phone or typing a message while riding is also distracting and unsafe.

The major problem identified is therefore the absence of a dedicated, centralised environment for monitoring a riding group. PackTrace is proposed to fill this gap by combining group management, live location sharing, map-based monitoring and safety functions in a single web application.

2.2 Solution Overview

PackTrace is a web-based group ride tracking platform. A rider registers and logs in, and then creates a ride group or joins an existing one. When the ride is active, the rider's browser obtains the GPS position of the device through the Geolocation API and sends it to the Django backend. The backend uses WebSocket communication, provided through Django Channels, to pass the position to the other members of the group, and their dashboards update the rider's marker on a Google Maps based map.

The dashboard presents the riders on the map together with ride information. If a rider stops sending updates, the system treats the rider as disconnected and keeps the last known location visible. If a rider moves away from the planned route, the system detects the deviation. A rider in an emergency can raise an SOS alert that is delivered to the group. When the ride ends, it is stored in the ride history, which the rider can view later. An administration interface allows the system to be managed.

The application is built with Python and Django on the server side, HTML, CSS and JavaScript on the client side, and a relational database for data storage. Exact

The following chapters describe the system design, implementation and testing.

2.3 Feasibility Study

A feasibility study evaluates whether the proposed system can be developed and operated with the available technology, cost and effort. For PackTrace the study considers technical, economic, operational and schedule feasibility.

Technical Feasibility

PackTrace uses mature and well-documented technologies. Python and Django provide the backend, authentication, database access and administration; Django Channels provides WebSocket support; the browser Geolocation API provides the device position; and the Google Maps Platform provides map display. All current browsers support the Geolocation API and WebSockets, and riders use their own smartphones, so no special hardware is needed. The technical implementation of the system is therefore feasible.

Economic Feasibility

Python, Django, Django Channels and the web technologies used are free and open source. The map service is a commercial platform, but it offers a usage tier suitable for development and demonstration, and its usage limits are an operating consideration for larger deployments. No hardware has to be purchased. The system is therefore economically feasible for an academic project and for small groups.

Operational Feasibility

The workflow of registering, logging in, joining a group and viewing the map is familiar from other web applications, so riders need little training. The dashboard presents the whole group on a single screen, which suits use by the group leader or by a rider at a stop. The system is therefore operationally feasible.

Schedule Feasibility

The system is modular. The core functions — authentication, ride groups and live tracking — can be completed first, followed by alerts, ride history and administration, which makes the work suitable for completion within the project period.

2.4 Requirement Specification

2.4.1 Functional requirements

Table 2.1 lists the functional requirements derived from the project objectives, use case diagram and interface screens. Status codes: E = interface evidence shown; O = reported as implemented.

ID	Requirement	Status
FR-01	The system shall allow a visitor to register with name, email, phone number, ride type and password, and to consent to share live location with the ride group.	E
FR-02	The system shall allow a registered user to log in with email and password and to log out.	E
FR-03	The system shall allow a rider to create a ride group and other riders to join it.	O
FR-04	The system shall obtain the rider's GPS position from the device and share it during an active ride.	O
FR-05	The system shall deliver location updates to the other members of the group in real time using WebSocket communication.	O
FR-06	The system shall display the riders of the group as markers on a map built with the Google Maps Platform.	O
FR-07	The system shall display the planned route and the path travelled by riders.	O
FR-08	The system shall treat a rider who stops sending updates as disconnected and show the rider's last known location.	O
FR-09	The system shall detect when a rider deviates from the planned route.	O
FR-10	The system shall allow a rider to raise an SOS alert that notifies the group.	O
FR-11	The system shall allow a ride to be ended and shall store it in the ride history.	O
FR-12	The system shall allow a rider to view previous rides.	O
FR-13	The system shall restrict ride data to authenticated users and shall provide an administration interface for authorised administrators.	O

Table 2.1 Functional requirements of PackTrace

2.4.2 Non-functional requirements

Attribute	Requirement
Performance	Location updates should reach other members' dashboards with low delay, and the map should remain responsive while markers update.
Usability	The interface should be simple, consistent and readable on desktop and mobile browsers.
Security	Only authenticated users may access ride data; passwords must be protected; location must be visible only to members of the same ride group.
Reliability	The system should tolerate temporary loss of connection and show the last known location of an affected rider.

Scalability	The architecture should allow more riders, groups and features to be added.
Availability	The application should be accessible through a browser whenever a ride is in progress.
Maintainability	The code should follow the Django project structure of models, views, URL routing and templates.
Privacy	Location should be shared only after the rider's consent and only within the ride group.

Table 2.2 Non-functional requirements

2.4.3 Hardware requirements

Component	Requirement
Computer / laptop (development, server, administration)	Dual-core processor, 4 GB RAM, 5 GB free disk space, display of 1366 × 768 or higher
Smartphone / device with GPS (rider)	Smartphone or tablet with a GPS receiver and a modern web browser; location services enabled
Internet connection	Mobile data or Wi-Fi connection for continuous exchange of location data during a ride

Table 2.3 Hardware requirements

2.4.4 Software requirements

The software versions used in the development environment are summarized in Table 2.4.

Category	Component	Version / remarks
Programming language	Python	3.10.11 (development environment)
Web framework	Django	5.2.16 (project environment)
Real-time communication	Django Channels (WebSocket)	4.3.2
Front end	HTML, CSS, JavaScript	Browser Geolocation API; WebSocket API
Map service	Google Maps Platform – Maps JavaScript API	Requires an API key; the earlier prototype used Leaflet (see Section 5.1)
Database	SQLite	SQLite3
Browser	Current Chrome, Edge or Firefox	Geolocation and WebSocket support
Development tools	VS Code, StarUML, Draw.io, GitHub	Coding, modelling and version control

Table 2.4 Software requirements

CHAPTER 3

LITERATURE REVIEW

This chapter reviews published work related to GPS-based tracking, real-time web communication, safety applications for road users, location privacy and geographic computation, which form the technical background of PackTrace. Only publications that can be traced through a journal, conference proceedings or publisher record are included, and they are presented with the most recent first.

3.1 GNN-RMNet: Graph Neural Networks and GPS Analytics for Driver Behavior and Route Optimization in Logistics

[Authors: E. A. Aldhahri, A. A. Almazroi, M. H. Alkinani, M. Alqarni, E. A. Alghamdi and N. Ayub (2025)] [1]

This study, published in PLOS ONE, proposes a hybrid deep learning model that analyses GPS trajectories and vehicle sensor data to identify risky driver behaviour and abnormal routes in logistics operations. The model combines a graph neural network with ResNet and MobileNet components so that both the spatial structure of the route and the temporal pattern of movement can be considered.

The work shows that GPS position data can be used not only to display where a vehicle is but also to decide whether it is following an expected route. It reports high accuracy in route anomaly detection and is designed for low-latency use on edge devices. However, it depends on vehicle sensor data and trained models, and its data came from one regional logistics network. It serves fleet operators rather than riding groups. For PackTrace the paper is relevant because it supports the idea of detecting deviation from a planned route from GPS data.

Key Aspects

- Route anomaly detection from GPS trajectories
- Hybrid deep learning model for logistics vehicles
- Use of GPS and vehicle sensor data
- Low-latency design suitable for edge devices
- Targeted at fleets, not at groups of riders

3.2 Real-Time GPS Tracking System for IoT-Enabled Connected Vehicles

[Authors: I. Moumen, N. Rafalia, J. Abouchabaka and M. Aoufi (2023)] [2] This paper, published in the E3S Web of Conferences, describes a real-time vehicle tracking system built from a microcontroller, a GSM module and a GPS module. The device reads the

vehicle position and sends it to a server, where it is stored and shown to users through a web interface.

The system demonstrates the basic pipeline used by tracking applications: acquisition of coordinates, transmission over a mobile network, storage, and visualisation in a web page. Its limitation for the present purpose is that every vehicle needs dedicated hardware, and the system is designed for monitoring vehicles by an operator, not for sharing positions among the members of a group. PackTrace follows the same pipeline but replaces the dedicated hardware by the rider's smartphone browser.

Key Aspects

- Real-time GPS tracking with web-based monitoring
- Hardware-based tracker: microcontroller, GSM and GPS modules
- Server storage and visualisation of positions
- No group or ride concept
- Dedicated hardware required for each vehicle

3.3 Towards a Sustainable City for Cyclists: Promoting Safety through a Mobile Sensing Application

[Authors: P. Boronat, M. Pérez-Francisco, C. T. Calafate and J.-C. Cano (2021)] [3]
This article, published in Sensors, presents a smartphone application for cyclists. The application regularly sends the cyclist's position to a server, and the server returns alarms when a potentially dangerous situation is detected. The work shows that periodic position reporting from an ordinary smartphone to a server is practical for road-safety applications.

The application is a native Android application and its alarms concern hazards between road users, not the coordination of a group. For PackTrace the study supports the choice of the smartphone as the tracking device and of a server that processes positions and returns alerts to users.

Key Aspects

- Smartphone application for cyclist safety
- Periodic position reporting to a server
- Server-generated alarms
- Native Android implementation
- Focus on traffic hazards rather than group coordination

3.4 Communicating and Displaying Real-Time Data with WebSocket

[Authors: V. Pimentel and B. G. Nickerson (2012)] [4]

This article, published in IEEE Internet Computing, examines the WebSocket protocol as a means of communicating and displaying real-time data in web applications and compares it with HTTP-based polling techniques. WebSocket provides a persistent, full-duplex connection, so that the server can send data to the browser as soon as it is available instead of waiting for the browser to ask.

The study shows that WebSocket reduces communication overhead and suits applications that update frequently. This is the technical reason for using WebSocket communication, through Django Channels, in PackTrace: rider positions change continuously and have to be delivered quickly to every member of the group.

Key Aspects

- Evaluation of WebSocket for real-time web data
- Comparison with HTTP polling approaches
- Lower overhead for frequent updates
- Persistent two-way connection
- Applicable to live location distribution

3.5 Accuracy of Phone-Based GPS Positioning

[Authors: P. A. Zandbergen and S. J. Barbeau (2011); P. A. Zandbergen (2009)] [5], [7]
Zandbergen and Barbeau evaluated the positional accuracy of assisted GPS in high sensitivity GPS-enabled mobile phones in the Journal of Navigation, and Zandbergen earlier compared assisted GPS, Wi-Fi and cellular positioning on a smartphone in Transactions in GIS. Both studies show that assisted GPS gives the best accuracy among the positioning methods available on a phone, but that errors increase in difficult surroundings such as indoors or among tall buildings.

For PackTrace these studies explain why the displayed position of a rider is approximate and why tolerance is needed when positions are compared with a route or with each other. They are also the basis for the limitation stated in Chapter 9 that GPS accuracy depends on the device and surroundings.

Key Aspects

- Empirical accuracy of phone-based positioning
- Comparison of assisted GPS, Wi-Fi and cellular positioning
- Accuracy decreases in difficult environments
- Results relate to the devices tested at that time
- Supports use of tolerances in route comparison

3.6 WreckWatch: Automatic Traffic Accident Detection and Notification with Smartphones

[Authors: J. White, C. Thompson, H. Turner, B. Dougherty and D. C. Schmidt (2011)] [6]

This article, published in Mobile Networks and Applications, describes WreckWatch, a smartphone application that detects traffic accidents using the phone's sensors and notifies emergency contacts and responders through web services. It demonstrates that a phone can act as an emergency-notification device without extra hardware.

The application targets automobile accidents and must manage false detections. PackTrace does not detect accidents automatically; instead, it lets a rider raise an SOS alert manually and notifies the group. The paper is relevant as an example of smartphone-based emergency notification, and automatic accident detection is listed as future scope.

Key Aspects

- Smartphone-based accident detection
- Automatic notification through web services
- No additional hardware needed
- False positives must be controlled
- Basis for future automatic detection in PackTrace

3.7 Location Disclosure to Social Relations: Why, When, & What People Want to Share

[Authors: S. Consolvo, I. E. Smith, T. Matthews, A. LaMarca, J. Tabert and P. Powledge (2005)] [8]

This paper from the ACM CHI conference reports a field study of how people decide whether to disclose their location to others. The findings show that people want control over who can see their location and in which situation, and that the identity of the requester and the reason for the request strongly influence the decision.

For PackTrace the findings support the design decision to share location only after the rider's explicit consent, only within a ride group chosen by the rider, and only during an active ride.

Key Aspects

- Field study of location disclosure
- Importance of user control over sharing
- Influence of who is asking and why
- Study carried out before modern smartphones
- Basis for consent-based sharing in PackTrace

3.8 Location Privacy in Pervasive Computing

[Authors: A. R. Beresford and F. Stajano (2003)] [9]

This article in IEEE Pervasive Computing analyses the privacy risks of location-aware services and introduces the idea of mix zones, in which users are anonymised while they move through a region. It provides a formal basis for reasoning about location privacy.

The approach aims at anonymous use of services, whereas PackTrace deliberately shares identified positions within a group. The paper is nevertheless relevant because it makes clear that stored location records are sensitive and should be limited, which is reflected in the security recommendations of Chapter 5.

Key Aspects

- Analysis of location privacy risks
- Mix-zone concept for anonymity
- Formal treatment of location privacy
- Anonymous services versus identified group sharing
- Motivation for limiting stored location data

3.9 Virtues of the Haversine

[Author: R. W. Sinnott (1984)] [10]

This short article in Sky and Telescope describes the haversine formula for computing the great-circle distance between two points specified by latitude and longitude. The formula is simple and remains numerically well behaved for short distances, which makes it suitable for comparing nearby positions.

It assumes a spherical Earth, so small errors remain in the computed distance. The haversine distance is a straight-line (great-circle) distance between two coordinates and is different from the road distance given by a routing service. It is described in Chapter 5 as the standard way of computing distance between coordinates.

Key Aspects

- Great-circle distance from latitude and longitude
- Numerically stable for short distances
- Assumes a spherical Earth
- Straight-line distance, not road distance
- Applicable to rider-to-rider and rider-to-route distances

3.10 Summary of the Literature

Author & Year	Title	Methodology	Advantages	Disadvantages
Aldhahri et al., 2025 [1]	GNN-RMNet: graph neural networks and GPS analytics for driver behavior and route optimization	Hybrid deep learning on GPS trajectories and sensor data	Accurate route anomaly detection; low latency	Needs sensor data and training; fleetoriented
Moumen et al., 2023 [2]	Real-time GPS tracking system for IoT-enabled connected vehicles	Microcontroller, GSM and GPS modules; server and web interface	Real-time web monitoring of positions	Dedicated hardware; no group concept
Boronat et al., 2021 [3]	Towards a sustainable city for cyclists	Smartphone app sending position to a server that returns alarms	Smartphone only; server-generated alerts	Native Android; traffic hazards, not group coordination
Pimentel & Nickerson, 2012 [4]	Communicating and displaying real-time data with WebSocket	Evaluation of WebSocket against HTTP polling	Low overhead for frequent updates	Not specific to location sharing
Zandbergen & Barbeau, 2011 [5]; Zandbergen, 2009 [7]	Accuracy of assisted GPS in mobile phones	Empirical accuracy comparison of positioning methods	Quantifies phone positioning accuracy	Errors in difficult environments; device-specific
White et al., 2011 [6]	WreckWatch	Sensor-based accident detection with webservice notification	Emergency notification without extra hardware	False positives; car accidents only
Consolvo et al., 2005 [8]	Location disclosure to social relations	Field study and interviews	Shows need for user control of sharing	Small, presmartphone study
Beresford & Stajano, 2003 [9]	Location privacy in pervasive computing	Analysis and mix-zone concept	Formal basis for location privacy	Aimed at anonymous use
Sinnott, 1984 [10]	Virtues of the Haversine	Great-circle distance formula	Simple and stable for short distances	Spherical Earth assumption

Table 3.1 Literature review of GPS-based tracking and related systems

The reviewed work can be broadly classified into three groups based on their contribution to GPS-based tracking, mobile safety, and the technical foundations required for real-time location applications. The first group, represented by Aldhahri et al. and Moumen et al., demonstrates how GPS data can be collected from vehicles, transmitted through communication networks, displayed on digital maps, and analysed to identify route-related anomalies. These studies establish the importance of continuous location monitoring for understanding vehicle movement and detecting situations in which a vehicle does not follow its expected path. Such approaches provide a useful foundation for PackTrace, particularly for implementing route monitoring and identifying deviations from a planned route during a group ride.

The second group, represented by Boronat et al. and White et al., demonstrates the potential of ordinary smartphones as practical location and safety-monitoring devices. Since modern smartphones are equipped with GPS or other location services, they can be used to determine a user's approximate position without requiring dedicated tracking hardware. These works also demonstrate how mobile devices can support safety-related functions by reporting a user's position and generating alerts when required. This is relevant to PackTrace because the system is designed to use the rider's own smartphone and web browser for location sharing, reducing the need for additional hardware or specialised tracking equipment.

The third group provides the technical and ethical foundations necessary for developing a real-time location-sharing system. WebSocket-based communication provides a mechanism for transmitting location updates with low latency between connected clients and the server, which is important when several riders need to view changing positions on a common map. The accuracy limitations of smartphone GPS also need to be considered because location readings may vary depending on environmental conditions, device capability, and signal availability. In addition, location-sharing applications must provide appropriate user control and transparency because continuous location information is sensitive personal data. The Haversine formula provides a simple mathematical method for calculating the approximate great-circle distance between two geographic coordinates and can therefore be used for estimating distances between riders or between a rider and a destination.

Although these studies provide important individual techniques and concepts, a practical gap remains in their combined application to group motorcycle riding. The reviewed works do not provide a lightweight, browser-based environment specifically organised around a riding group in which multiple riders can participate in the same ride, view one another on a common map, and monitor the progress of the group. In particular, there is a need for a system that combines group creation and joining, real-time location sharing, handling of disconnected riders through last-known location information, planned-route monitoring, route deviation detection, and an SOS mechanism within a single interface.

PackTrace addresses this identified gap by combining established GPS, web mapping, real-time communication, and distance-calculation techniques into a single web application designed for group ride coordination. The system allows riders to participate in a shared ride and view the locations of group members on a common map while the ride is in progress. It also incorporates mechanisms for monitoring route deviation and dealing with temporary loss of connection, while providing an SOS feature for emergency situations. Therefore, rather than introducing a completely new GPS or communication technique, PackTrace focuses on integrating existing technologies into a practical, lightweight, browser-based solution for coordinated group riding.

CHAPTER 4

DIAGRAMS

4.1 Data Flow Diagram

A Data Flow Diagram (DFD) represents the flow of data within a system. It shows how information moves between processes, data stores and external entities. In PackTrace, the DFD illustrates how registration details, ride group details, GPS locations, SOS requests and ride history requests are processed, stored and returned to riders and administrators.

4.1.1 DFD Level 0

The Level 0 diagram, also known as the context diagram, represents PackTrace as a single process, the GPS-Based Group Ride Tracking System, and shows how it interacts with four external entities: the User / Rider, the Admin, the GPS service / GPS device and the map service. The rider sends registration and login details, ride group details, live GPS location, SOS alerts, requests to view the group and ride history requests. The system returns login confirmation, group details, live rider locations, alerts and notifications, and ride history. The administrator sends login credentials and management requests and receives generated reports and system data. The GPS source supplies location data, and map requests and map data are exchanged with the map service.

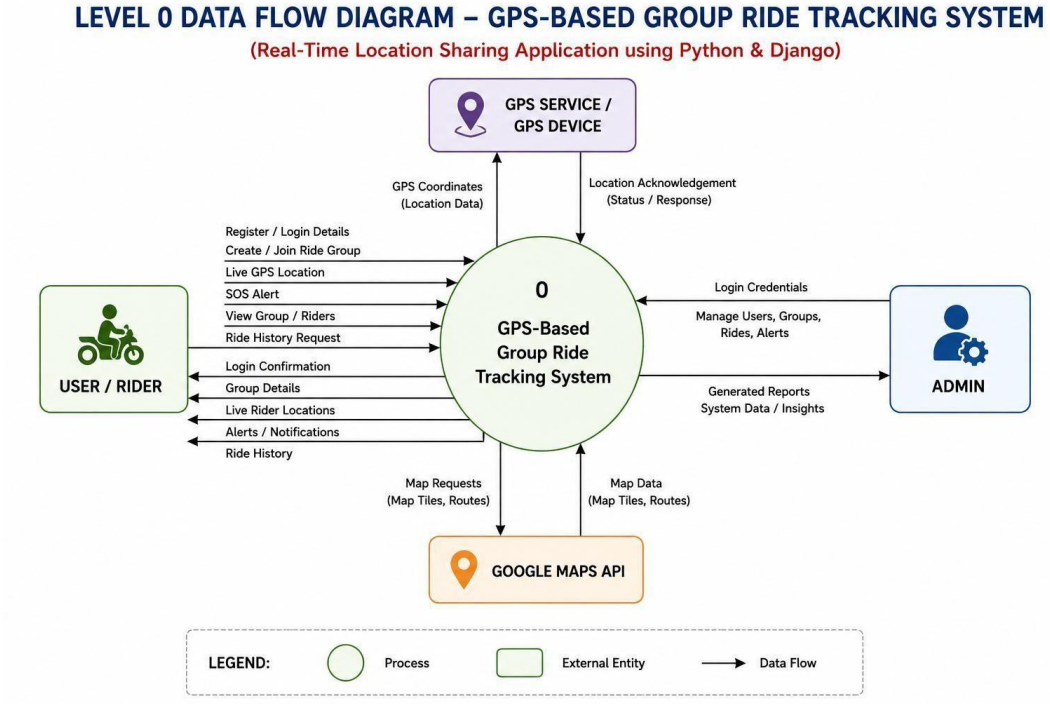


Figure 4.1 DFD Level 0

4.1.2 DFD Level 1

The Level 1 diagram expands the main process into its functional modules. The user module consists of five processes. Process 1.0, User Login / Registration, saves and verifies user details in the User Database (D1). Process 2.0, Create / Join Ride Group, stores group information in the Group Database (D2) and returns a group created or joined confirmation. Process 3.0, Live GPS Tracking, receives GPS coordinates, stores the live location in the GPS Location Database (D3) and returns the live location of group members. Process 4.0, SOS & Emergency Alert, stores alert information in the Alert Database (D4) and returns an alert notification and acknowledgement. Process 5.0, Ride History, retrieves ride history from the Ride History Database (D5) in response to a history request.

The administration module contains Process 6.0, Admin Management, which allows the administrator to manage users, manage ride groups, monitor live riders, manage SOS alerts, manage ride history and generate reports, using all the system databases D1 to D5. The report-generation and monitoring functions shown here are part of the design; the administration functions actually implemented are described in Section 5.1.

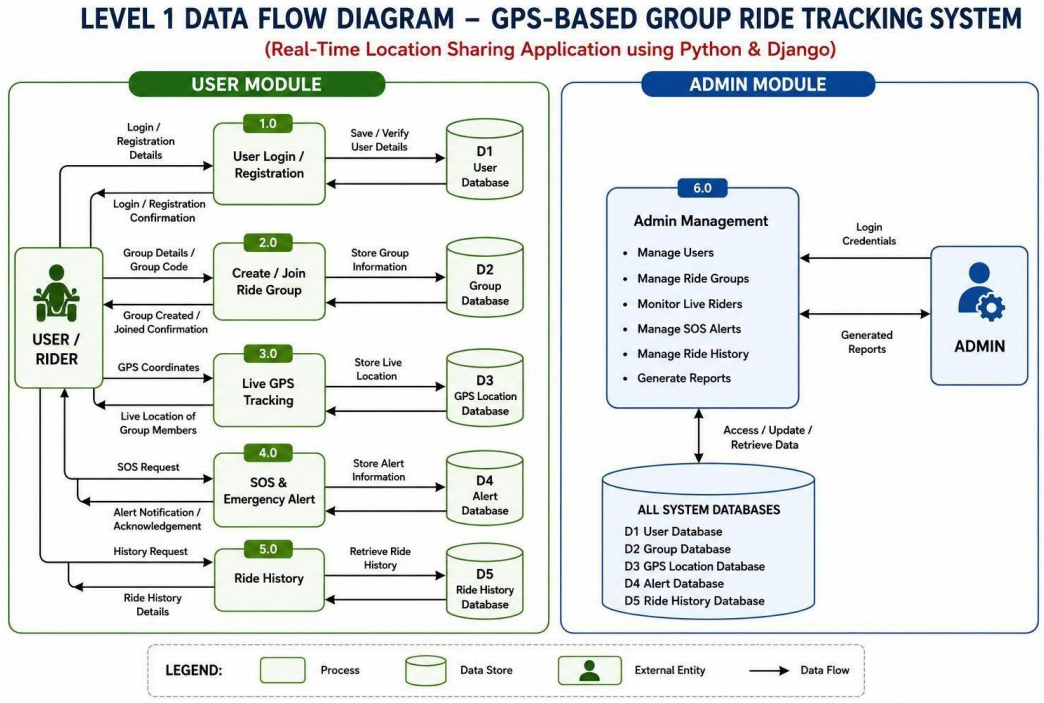


Figure 4.2 DFD Level 1

4.2 Use Case Diagram

The use case diagram shows the functions available to the two actors of the system. The Rider / User can register, log in, manage the profile, create a ride group, join a ride group, share live GPS location, view the live map, track group members, send an SOS alert, receive notifications, view ride history and leave a ride group. Sharing a location, viewing the map and tracking group members include the use case Update Live Location; sending an SOS

alert extends to Alert Group Members; viewing ride history includes Retrieve Ride History; and registration and login include Authenticate User, which includes Validate Credentials.

The Admin can log in, manage users, manage ride groups, monitor live riders, manage SOS alerts, view ride history, generate reports and manage system settings. Because the administration functions in the diagram are part of the design, the implemented administration functions are listed separately in Section 5.1.

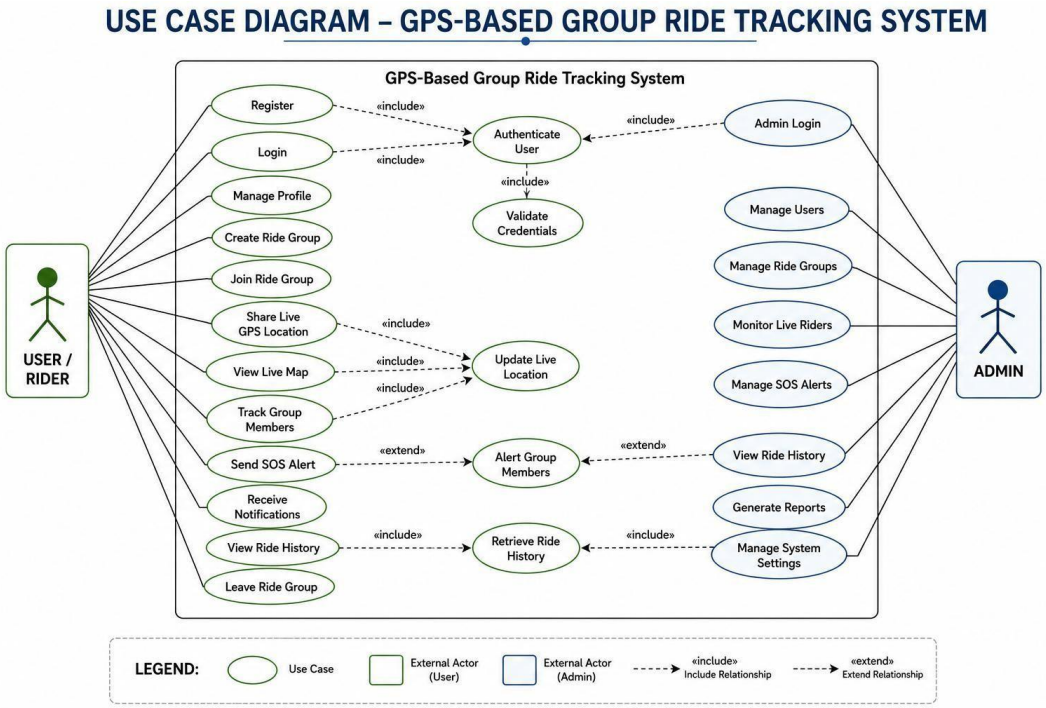


Figure 4.3 Use Case Diagram

4.3 Class Diagram

The class diagram represents the main classes of the system, their attributes and operations, and the relationships between them. A User creates ride groups, joins groups through group members, shares GPS locations, triggers SOS alerts and has a ride history. A RideGroup contains one or more GroupMembers and records RideHistory. An SOSAlert generates a Notification. The Admin manages users and ride groups, monitors SOS alerts and generates reports.

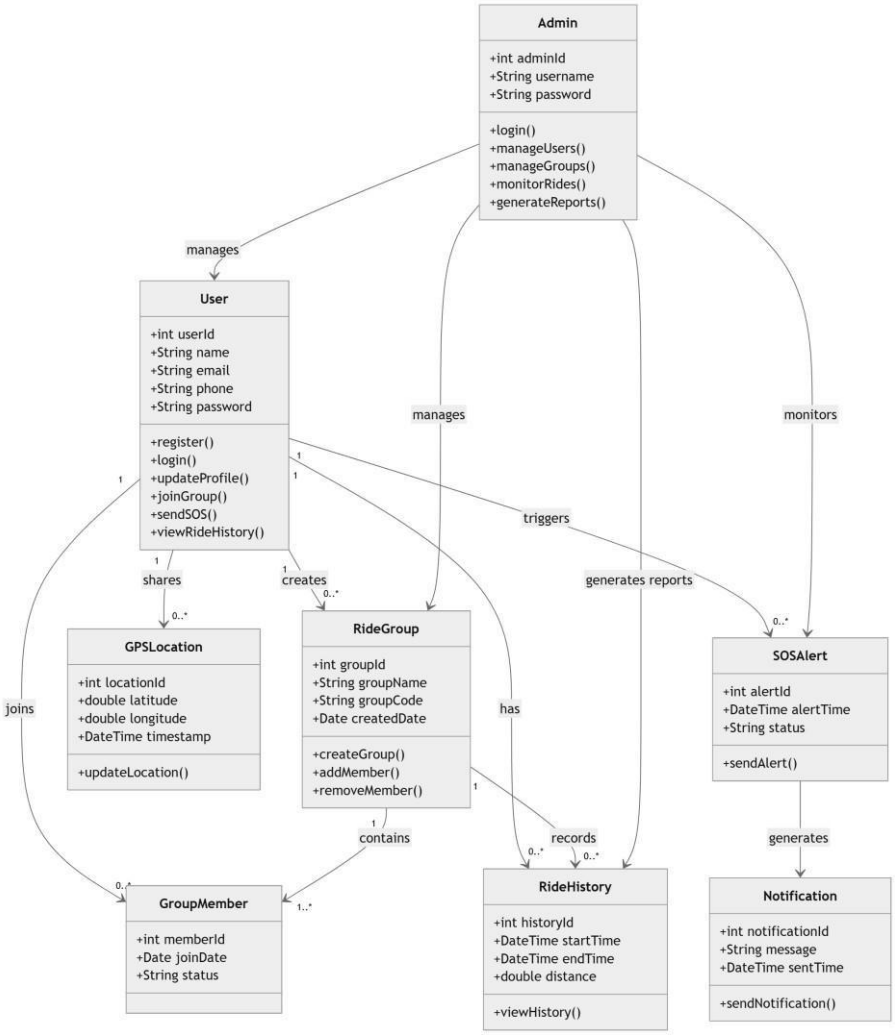


Figure 4.4 Class Diagram (design-stage conceptual model)

4.4 State Diagram

The state diagram shows the connection states of a rider during a ride. The rider is initially idle, starts the ride and connects, and then shares location while connected. If the connection is lost, the rider is shown as disconnected and the last known location remains visible to the group. When the rider reconnects, location sharing resumes; when the ride ends, the rider reaches the ride ended state.

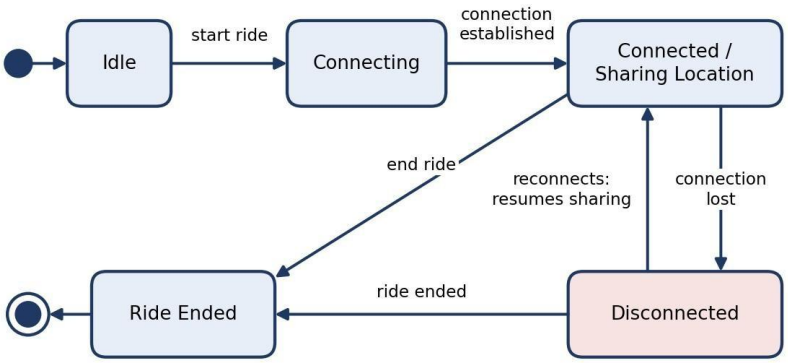


Figure 4.5 State Diagram

4.5 Activity Diagram

The activity diagram shows the workflow of PackTrace from registration or login to the storing of the ride history. The rider registers or logs in, creates or joins a ride group and starts the ride. The device GPS location is captured and sent to the server, the server updates the group view, and the map shows all riders. During the ride the system monitors for SOS alerts and for riders who disconnect; a disconnected rider is marked and the last known location is shown. When the ride ends, the ride history is stored.

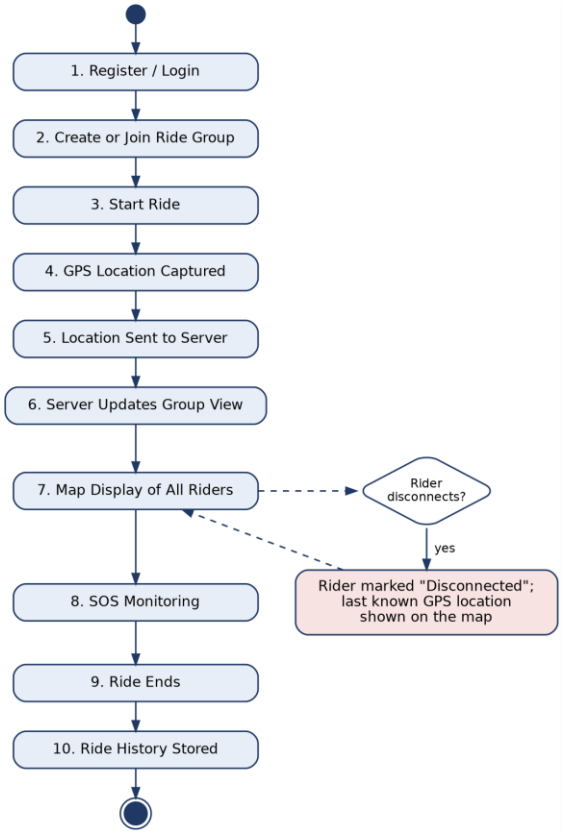


Figure 4.6 Activity Diagram

4.6 Sequence Diagram

The sequence diagram shows the order of messages between the User (Rider), the Web / Mobile Interface, the Django Server, the Database and the GPS API / Google Maps. It

contains five interaction groups. In user authentication, the rider submits registration or login details, the Django server verifies the user in the database and returns an authentication response. In group management, the rider creates or joins a group, and the server stores the group information and returns a confirmation. In live location sharing, the server receives the location (latitude and longitude), saves it in the database and returns the updated location, which is shown on the map. In the SOS alert, the server saves the alert information, returns the alert status and notifies the group members. In ride history, the server retrieves the history from the database and returns it for display.

In the browser implementation the coordinates themselves are produced by the device through the Geolocation API; the GPS API / Google Maps participant of the diagram stands for the location source and map service.

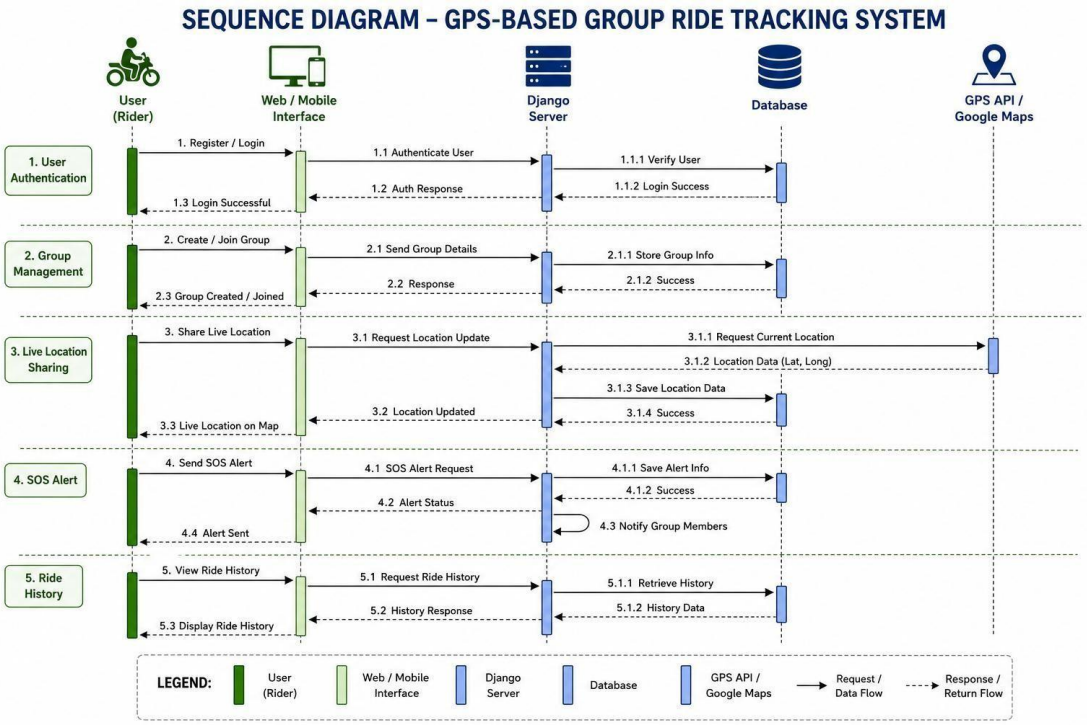


Figure 4.7 Sequence Diagram

4.7 Entity Relationship Diagram

The entity relationship diagram presents the conceptual relationships among the main PackTrace data entities. The implemented Django application models are described in Section 4.8.

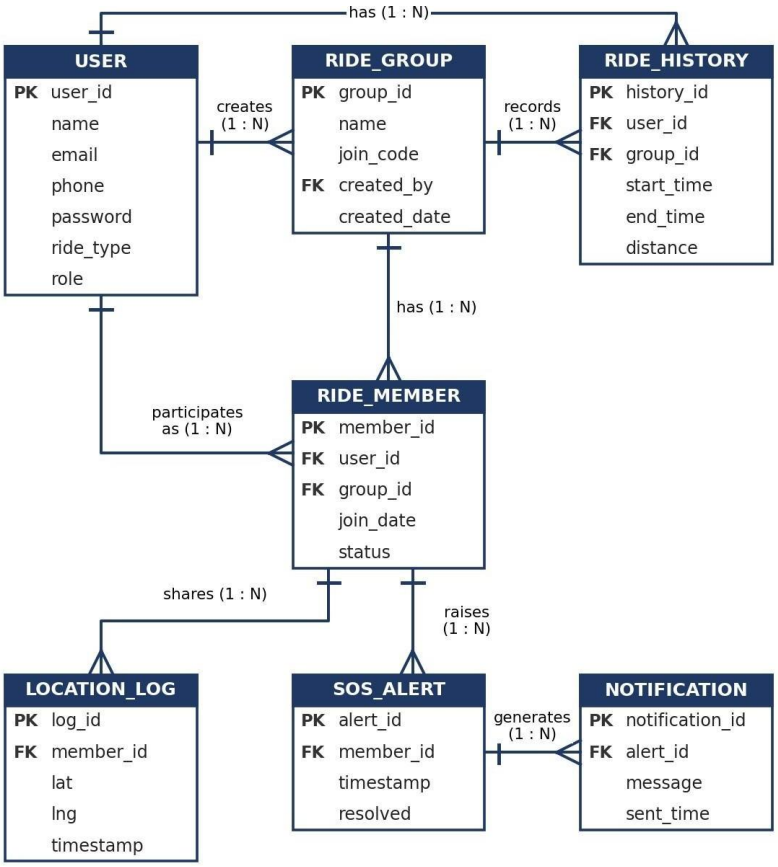


Figure 4.8 Entity Relationship Diagram

4.8 Database Design

The tables below present the logical database design for PackTrace. Django's built-in authentication tables, including User, Group and Permission, are maintained by Django. The implemented PackTrace application defines five models: RiderProfile, RideGroup, RideMember, LocationLog and SOSAlert. RIDE_HISTORY and NOTIFICATION are conceptual design entities, not separate Django models unless they are present in the source code.

Table 4.1 USER table (application user)

Field Name	Data Type	Key	Description
user_id	Integer	PK	Unique identifier of the user
name	String		Full name entered at registration
email	String		Email address used to sign in
phone	String		Phone number entered at registration
password	String		Password, stored in hashed form
ride_type	String		Vehicle type selected at registration
role	String		Distinguishes a rider from an administrator

Table 4.2 RIDE_GROUP table

Field Name	Data Type	Key	Description
group_id	Integer	PK	Unique identifier of the ride group
name	String		Name of the ride group
join_code	String		Code used by riders to join the group
created_by	Integer	FK	User who created the group
created_date	Date		Date of creation

Table 4.3 RIDE_MEMBER table

Field Name	Data Type	Key	Description
member_id	Integer	PK	Unique identifier of the membership
user_id	Integer	FK	Rider
group_id	Integer	FK	Ride group
join_date	Date		Date of joining
status	String		Status of the member

Table 4.4 LOCATION_LOG table

Field Name	Data Type	Key	Description
log_id	Integer	PK	Unique identifier of the location record
member_id	Integer	FK	Member to whom the location belongs
lat	Decimal		Latitude
lng	Decimal		Longitude
timestamp	Date-time		Time of the location record

Table 4.5 SOS_ALERT table

Field Name	Data Type	Key	Description
alert_id	Integer	PK	Unique identifier of the alert
member_id	Integer	FK	Member who raised the alert
timestamp	Date-time		Time of the alert
resolved	Boolean		Whether the alert has been resolved

Table 4.6 RIDE_HISTORY table

Field Name	Data Type	Key	Description
history_id	Integer	PK	Unique identifier of the ride record
user_id	Integer	FK	Rider
group_id	Integer	FK	Ride group
start_time	Date-time		Start time of the ride
end_time	Date-time		End time of the ride
distance	Decimal		Distance covered

Table 4.7 NOTIFICATION table

Field Name	Data Type	Key	Description
notification_id	Integer	PK	Unique identifier of the notification
alert_id	Integer	FK	Alert that generated the notification
message	String		Text of the notification
sent_time	Date-time		Time the notification was sent

CHAPTER 5

METHODOLOGY

This chapter describes the methodology of PackTrace, including its major modules, algorithms, system architecture, and a case study demonstrating the working of a group ride. It explains how the system handles user registration and authentication, ride group creation and joining, real-time GPS tracking, route monitoring, SOS alerts, ride history, and administrative functions. The chapter also describes the sequence of operations involved in creating a ride, tracking group members, monitoring their movement, and completing the ride.

5.1 Module Descriptions

Table 5.1 presents the major modules of PackTrace and their functions within the system. The modules include user registration and authentication, ride group management, real-time GPS tracking, route monitoring, SOS alerts, ride history, and administration. Each module contributes to the overall functionality of PackTrace by supporting group ride coordination, location sharing, safety monitoring, and system management.

Table 5.1 Modules of PackTrace and available evidence

Module	Status	Evidence in this report
Registration and login	Implemented; interface shown	Figures 5.2 and 5.3
Ride / group creation and joining	Implemented	Figure 5.4
Live GPS tracking and WebSocket updates	Implemented	Code and screenshots
Map display (Google Maps Platform)	Implemented	Figure 5.5
SOS and notifications	Implemented	Figure 5.7
Ride history	Implemented	Figure 5.8
Administration	Implemented	Figure 5.9

5.1.1 User interface design

The wireframes illustrate the user interface design of the PackTrace web application. The design includes the main pages required for group ride tracking, including the Landing Page, Login Page, Dashboard Overview, Live Tracking, Group Management, and Alerts Page. The Landing Page introduces PackTrace and its purpose of providing real-time GPS tracking for group rides. The Login Page provides access to registered users, while the Dashboard provides an overview of active rides, group members, and important ride information.

The Live Tracking interface provides a map-based view of riders and their locations during a ride. The Group Management page allows users to create and manage riding groups, while the Alerts Page provides a centralized view of important ride-related

alerts. These wireframes represent the interface structure of PackTrace and provide a visual representation of the application's main screens and navigation.

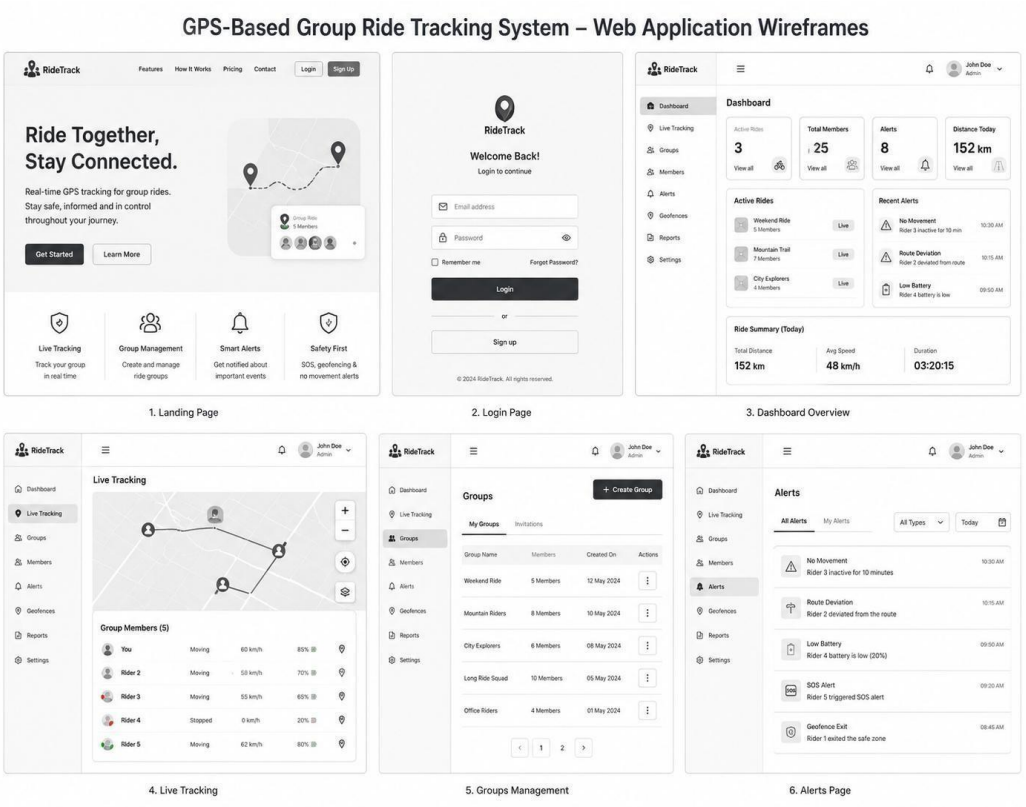


Figure 5.1 Web Application Wireframes

5.1.2 Registration and login module

The registration page collects the full name, email address, phone number, ride type, password and password confirmation, and contains a checkbox by which the rider agrees to share live location with the ride group during active rides, in accordance with the terms and privacy policy. The login page asks for the email address and password and offers a "keep me signed in" option, a "forgot password" link and a link to create an account.
Authentication is handled by the Django backend using the Django authentication and session framework. The application uses Django's built-in User model for authentication, while rider-specific information such as phone number, ride type and role is maintained through the RiderProfile model. Passwords are stored using Django's password-hashing mechanism, and logout terminates the authenticated session.

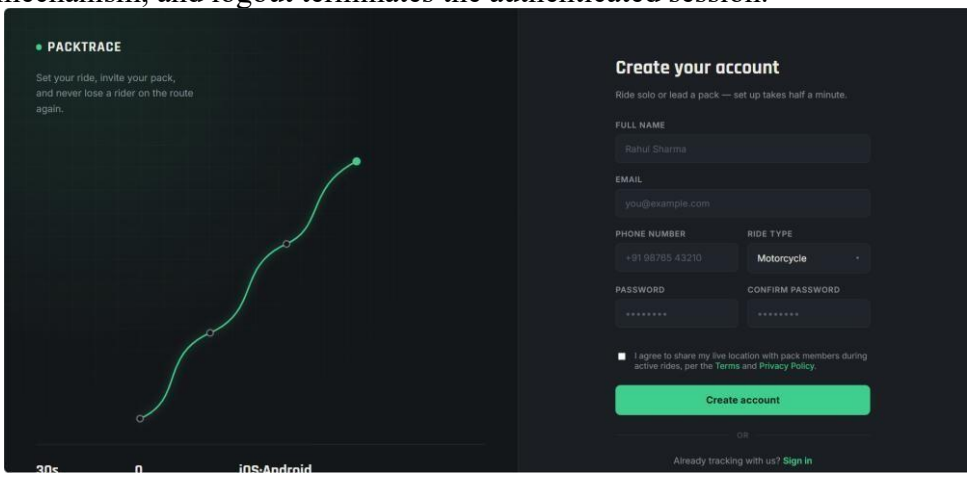


Figure 5.2 Register

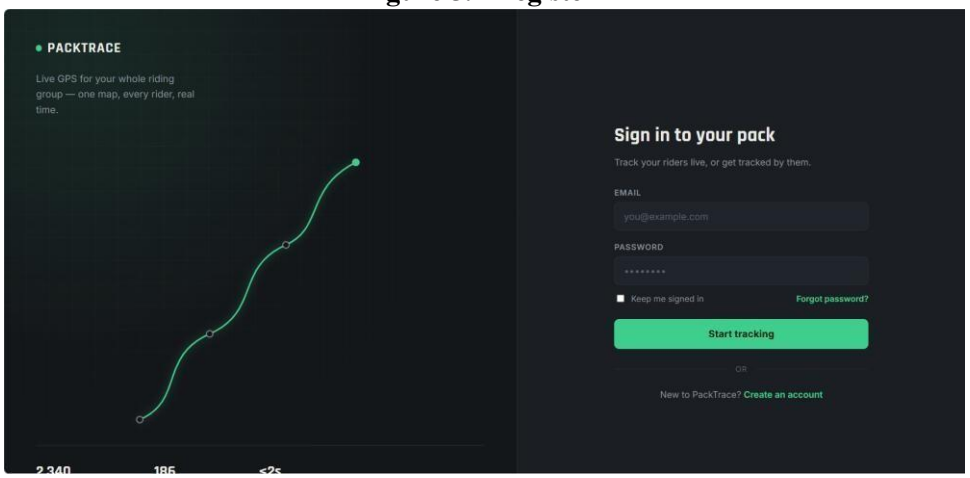


Figure 5.3 Login

5.1.3 Ride creation and joining module

A rider can create a ride group, and other riders can join it using a unique join code generated for the group. The rider who creates the group is recorded as the creator and acts as the ride leader. During group creation, the application stores the group name and creator information, along with ride details such as the destination and its geographic coordinates when provided. A rider joining an existing group provides the corresponding join code, and the application validates the code before adding the rider as a member of the group. Each user can have only one membership record for a particular group. The application does not define a fixed maximum number of riders in the group.

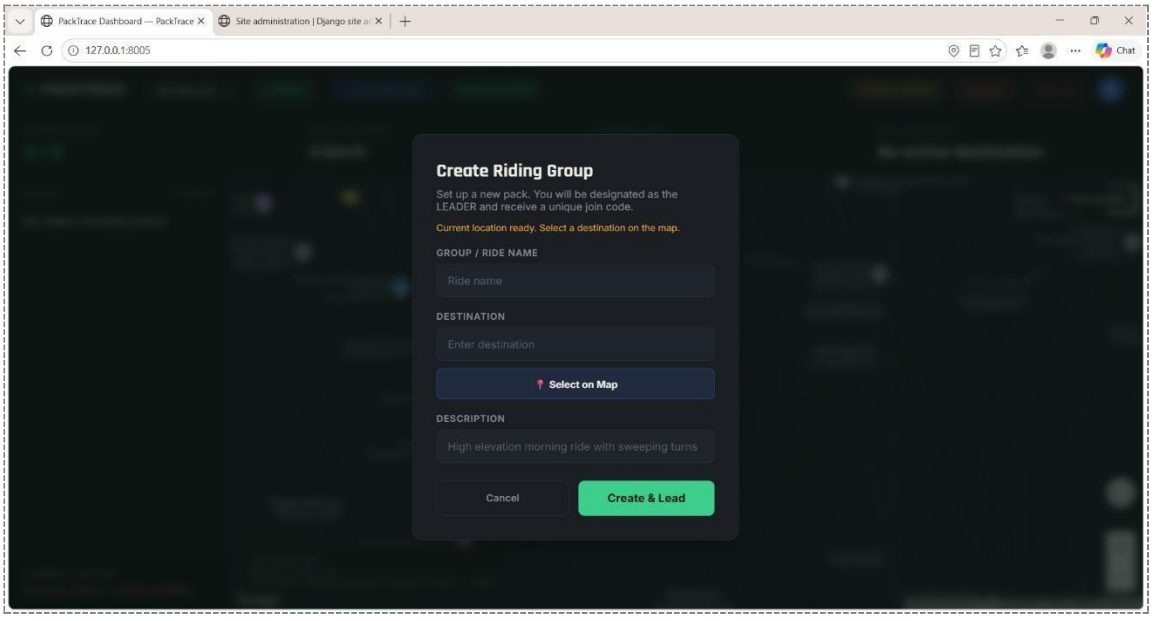


Figure 5.4 Create / Join ride screen

5.1.4 Live GPS tracking module

The rider's browser obtains the device position through the HTML5 Geolocation API after the rider grants permission. PackTrace uses navigator.geolocation.watchPosition() with high-accuracy positioning enabled, a timeout of 60 seconds and a maximum

cachedposition age of 5 seconds. The latitude, longitude, speed, accuracy and heading obtained from the device are sent to the Django backend through a WebSocket connection.

The WebSocket connection is established using the route /ws/rides/<group_id>/, with RideTrackingConsumer from tracker/consumers.py handling the connection. The consumer verifies that the authenticated user is a member of the ride group before accepting the connection. Location messages use the location_update message type and contain latitude and longitude together with speed, accuracy, heading and timestamp information.

When a location update is received, the consumer validates the coordinates, stores the location in the LocationLog database model and broadcasts a location_update message to the other connected members of the ride group through the Django Channels channel group. The dashboard receives these WebSocket messages and updates the corresponding rider marker, speed information and location on the Google Maps display.

The application also contains HTTP location-update functionality as a fallback when the WebSocket connection is unavailable. Therefore, the implementation primarily uses Django Channels WebSockets for real-time GPS updates, with an HTTP fallback available in the dashboard.

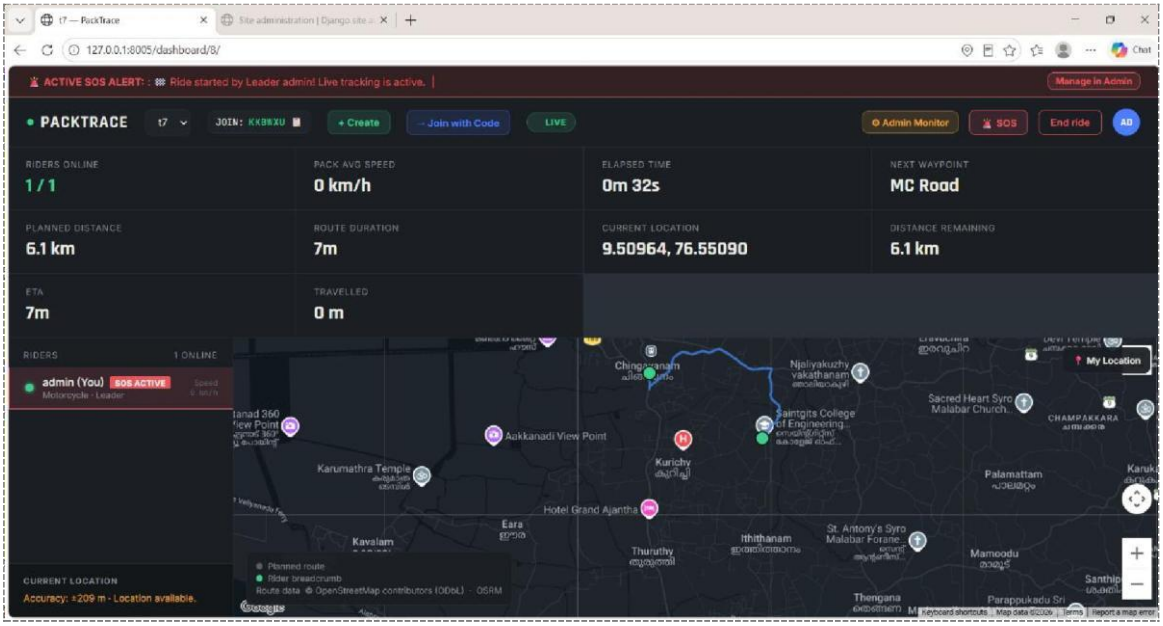


Figure 5.6 Live Google Maps view

5.1.5 Map display module

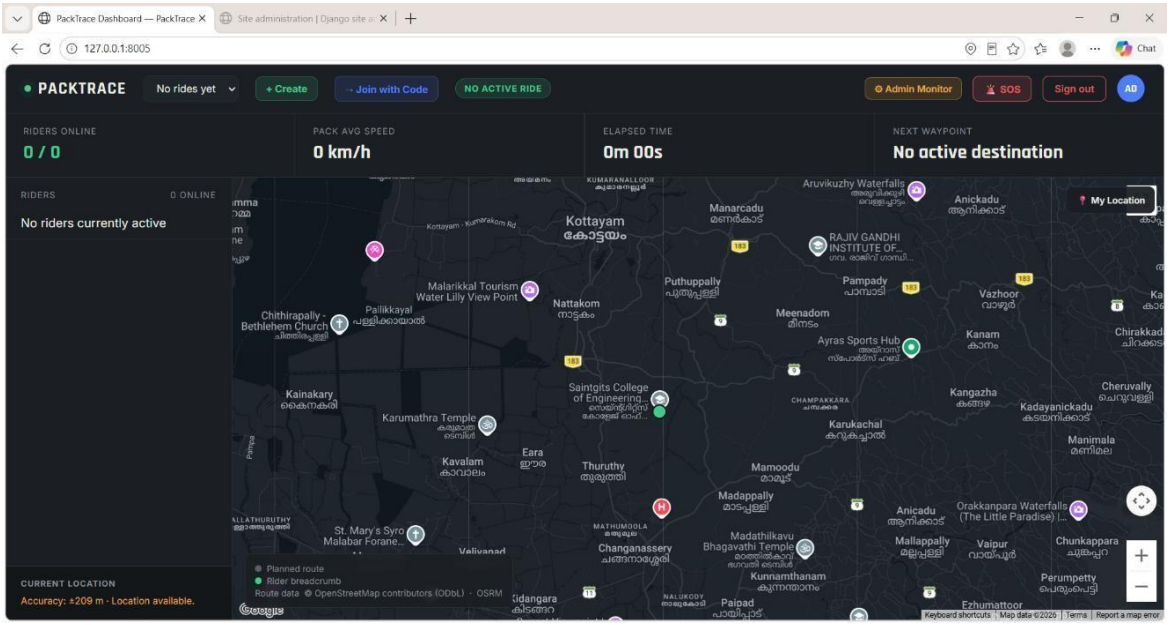


Figure 5.5 Dashboard layout

The figure shows the PackTrace dashboard layout with the PackTrace header, ride selector, Create and Join with Code controls, ride status indicator, and SOS and administration controls. The statistics strip displays riders online, pack average speed, elapsed time and the next waypoint. The left panel contains the rider list and current location information, while the main area displays the Google Maps-based map with map controls, rider/location markers and the legend for the planned route and rider breadcrumb. The figure represents the dashboard interface of the final application. The values shown in the screenshot, such as the rider count, speed, elapsed time and destination status, represent the state of the application at the time the screenshot was captured and should not be interpreted as fixed system values.

The final map provider of PackTrace is the **Google Maps Platform**, using the **Maps JavaScript API**. The dashboard integrates Google Maps to display the riders' current positions and the selected ride destination. Each rider is represented by a map marker, and the marker position is updated when new location information is received from the backend. The dashboard also displays the planned route and rider location information associated with the active ride.

The map is initialized using the Google Maps API key configured for the application. The selected destination is represented using its latitude and longitude, allowing the map to display the destination together with the riders' current positions. The earlier prototype used Leaflet with an OpenStreetMap-based CARTO basemap and sample data; that prototype is not the final map implementation. The final PackTrace dashboard uses Google Maps for the map display and location visualization

Table 5.2 Map implementation in the final application

Aspect	Final implementation
Map provider	Google Maps Platform
API used	Maps JavaScript API
Marker handling	Google Maps Marker objects are created for each rider. Existing markers are updated with the rider's latest latitude and longitude. Disconnected riders are displayed with reduced marker opacity. Rider markers can be clicked to display the rider's name and speed.
Rider location updates	GPS coordinates are received through the browser Geolocation API. The coordinates are sent through the Django Channels WebSocket connection using location_update messages. The dashboard receives the updates and moves the corresponding Google Maps marker. An HTTP location-update endpoint is also available as a fallback when WebSocket communication is unavailable.
Route / breadcrumb display	The planned route is displayed using a Google Maps Polyline. Each rider also has an individual Polyline that records successive positions and displays the rider's breadcrumb trail. The planned route is shown in blue, while the rider breadcrumb uses the rider's assigned colour.
Destination display	The selected destination is stored using its latitude and longitude and displayed on the map using a Google Maps marker. The destination name is also shown in the dashboard's waypoint/statistics area.
Map interaction	The map supports zooming, panning, fullscreen control and a **My Location** button. Riders can be selected from the rider list, and clicking a rider's map marker opens an information window showing the rider's name and speed. During ride creation, the user can click on the map to select a destination.
Route calculation	PackTrace uses an **OSRM routing service**, not Google Directions API, to calculate the road route. The Django backend sends the starting and destination coordinates to the OSRM routing service and receives the route geometry, distance and duration. The resulting route is then displayed on Google Maps using a Polyline.

5.1.6 Disconnect detection module

A rider who stops sending location or connection updates is treated as disconnected, and the rider’s last known location remains available to the group. PackTrace uses two

mechanisms for disconnect detection. First, when a rider's WebSocket connection is closed, the disconnect() method of Ride Tracking Consumer marks the corresponding ride member as disconnected and broadcasts the connection-status change to the other riders. Second, the backend periodically checks the last seen value of active riders. If the time since the last update exceeds the configured disconnect threshold of 45 seconds, the rider is marked as DISCONNECTED and a rider disconnected event containing the last known coordinates is broadcast to the ride group. The last recorded location is retained and can therefore remain visible on the dashboard even after the rider is disconnected. When the rider reconnects or sends a new GPS update, PackTrace changes the rider's status back to ACTIVE and updates the last seen value.

5.1.7 Route deviation detection module

PackTrace detects when a rider moves away from the planned route by comparing the rider's current GPS coordinates with the stored points of the planned route. The backend uses the Haversine distance formula to calculate the distance between the rider's current position and each route point, and selects the minimum calculated distance. If the minimum distance is greater than the configured route-deviation threshold, the rider is marked as having deviated from the planned route and a route-deviation notification is generated for the ride group. The threshold is defined by the ROUTE_DEVIATION_THRESHOLD_METERS constant in the Django settings and is set to 500 metres. When the rider returns within the permitted distance, the route deviation status is cleared. The route-deviation check is performed when a new location update is processed by the backend.

5.1.8 SOS and notification module

A rider in an emergency can raise an SOS alert using the SOS control in the dashboard. When the alert is submitted, the Django backend verifies the rider and the active ride membership, stores the SOS alert together with the rider's current location and message, and assigns the alert an active status. The alert is then made available to the other members of the ride group and to the administrator. The receiving riders are notified of the emergency and can identify the rider who raised the alert and the associated location information. The administrator can monitor active SOS alerts and manage their status, including resolving an alert.

The SOS information is maintained in the SOSAlert database model. The project design also includes a notification record associated with the alert. PackTrace does not claim SMS, email, mobile push notifications or audio recording as part of the current implementation; these can be considered future enhancements.

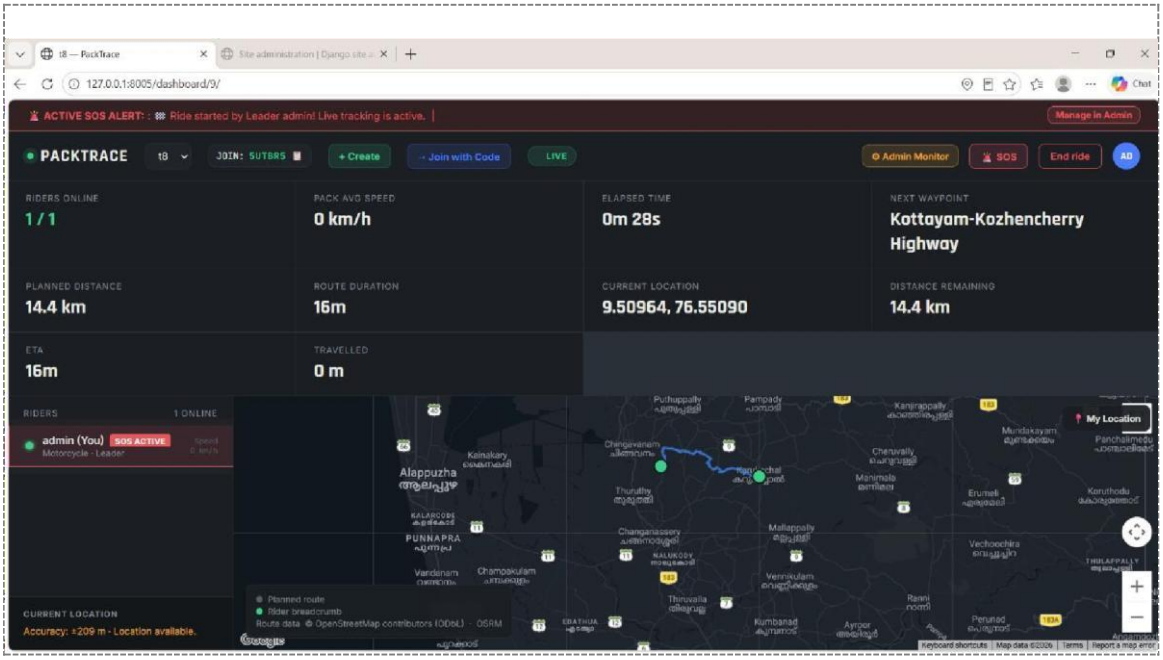


Figure 5.7 SOS alert and notification screen

5.1.9 Ride history module

When a ride is completed, its details are stored in the database. The rider can access the Ride History page to view all previously completed rides. The information displayed, such as the pack name, start location, destination, planned distance, travelled distance, ride duration, rider's role, ride status, and pack members, is retrieved from the application's database models and rendered using Django templates.

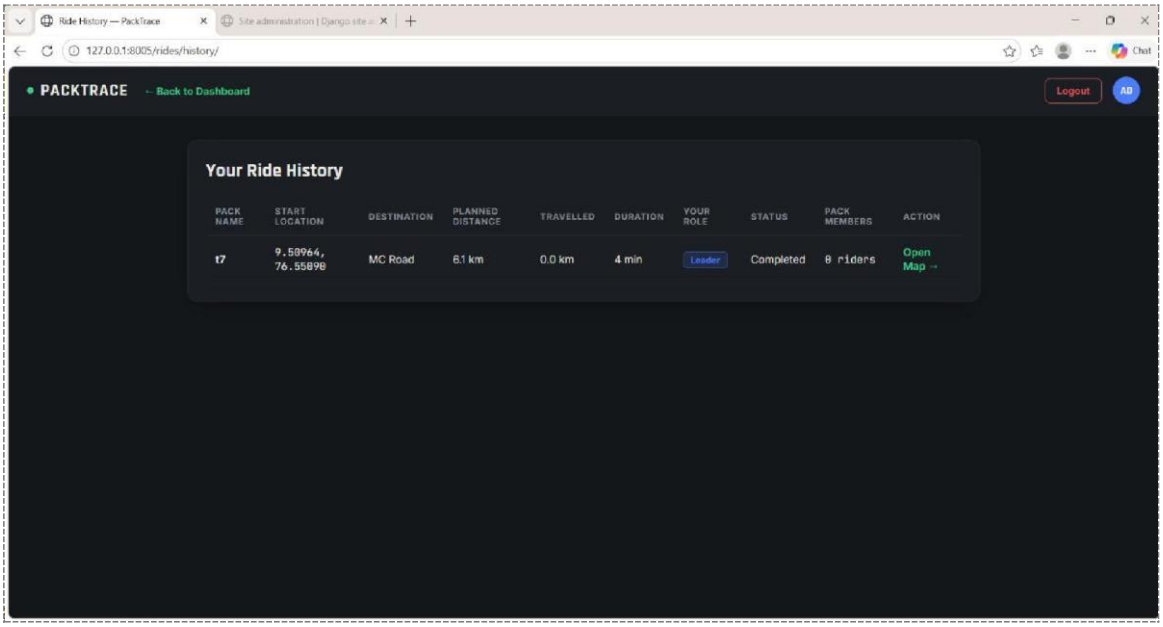


Figure 5.8 Ride history screen

5.1.10 Administration module

Administration is provided through the Django Administration site (/admin/). The administrator can manage application data through the built-in Django admin interface. Based on the implemented models, the administrator can manage Users, Groups, Rider

Profiles, Ride Groups, Ride Members, Location Logs, and SOS Alerts. These management functions include viewing, adding, editing, and deleting records.

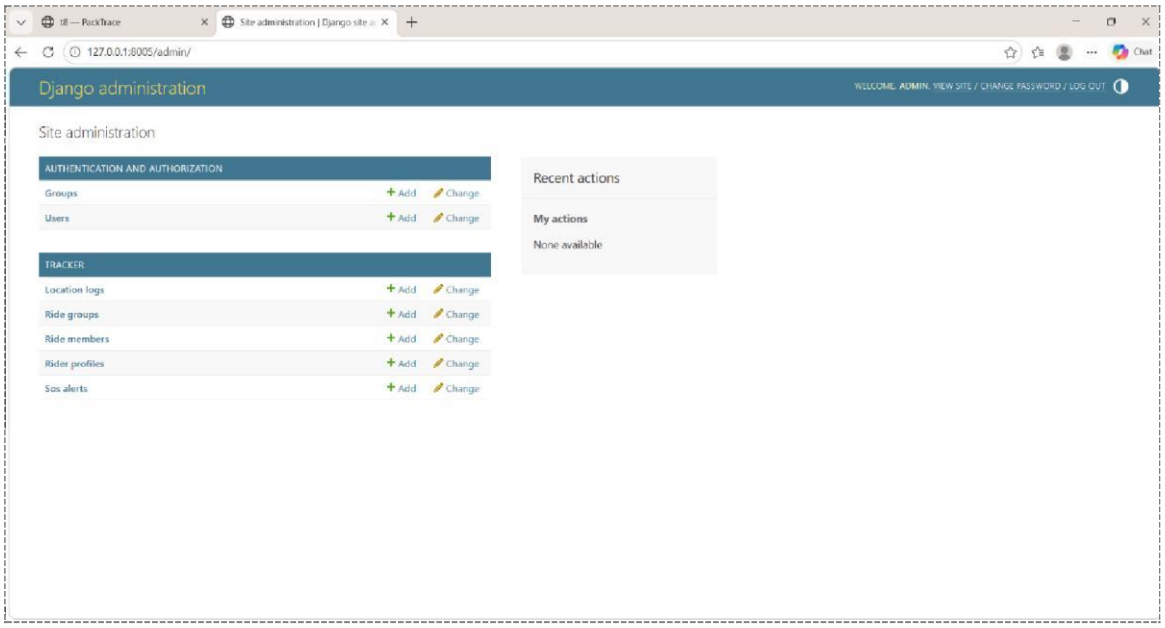


Figure 5.9 Administration interface

5.2 Algorithms

5.2.1 GPS location update

```
ALGORITHM 1: GPS location update
INPUT  : authenticated rider, active ride group
OUTPUT : updated rider marker on every member dashboard


1. Request permission to access the device location.
2. IF permission is denied THEN show a message and STOP.
3. Obtain latitude, longitude and timestamp from the Geolocation API.
4. Send the location to the Django backend.
5. Backend: validate the session and the data.
6. Backend: store the location and pass it to the other members of the group.
7. Each dashboard: move the rider's marker on the map.
8. Repeat from step 3 while the ride is active.
```

5.2.2 Haversine distance

The great-circle (straight-line) distance between two points given by latitude and longitude can be computed with the haversine formula. With latitudes φ1 and φ2, longitudes λ1 and λ2, and the Earth's mean radius R = 6,371 km:

```
a = sin²(Δφ / 2) + cos φ1 × cos φ2 × sin²(Δλ / 2) c = 2 × atan2(√a, √(1 − a)) d
= R × c


where Δφ = φ2 − φ1 and Δλ = λ2 − λ1 (in radians)
```

PackTrace uses two different types of distance calculations depending on the application feature. Straight-line (geodesic) distances are calculated using the Haversine formula for estimating the travelled distance, determining the nearest route waypoint, calculating the remaining distance, and detecting route deviation. The planned journey distance,

Road distance is obtained from the OSRM routing service, which follows the road network. The application distinguishes between straight-line (Haversine) distance and road distance, using each where appropriate.

5.2.3 Route and breadcrumb tracking

The rider's travelled path (breadcrumb trail) is constructed from successive GPS location updates. Each new location is stored in the LocationLog model and appended to the rider's path. On the map, the travelled path is drawn as a polyline connecting recorded GPS points, while the planned route is displayed using route geometry obtained from the OSRM routing service. This allows comparison of the planned route with the rider's actual path.

5.2.4 Disconnect detection

```
ALGORITHM 2: Disconnect detection (generic form)

1.          For each member of an active ride group:
2.          determine whether updates from the member are still arriving
3.          IF no update is arriving THEN
4.          mark the member as disconnected
5.          keep the last known location visible on the map
6.          ELSE IF the member was disconnected THEN
7.          mark the member as connected again

Mechanism and timeout used by PackTrace: see Section 5.1.6
```

5.2.5 Route deviation check

PackTrace implements route deviation detection using the Haversine formula. The rider's current GPS coordinates are compared with the waypoints of the planned route, and the minimum straight-line distance is calculated. A rider is considered to have deviated when this distance exceeds the configurable threshold stored in the constant DEVIATION_THRESHOLD, whose value is obtained from the application setting ROUTE_DEVIATION_THRESHOLD_METERS (default: 500 meters). The application does not use Google Maps route geometry or routing services for deviation detection.

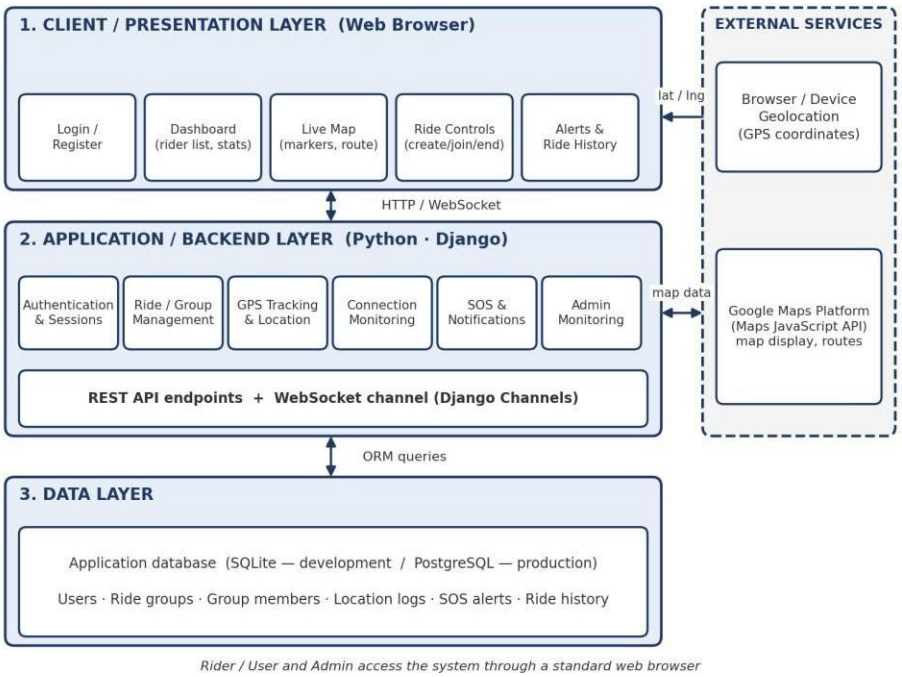
5.2.6 Authentication workflow

The credentials are submitted; the server looks up the user and checks the supplied password against the stored password hash; if the check succeeds a session is created and the rider is directed to the dashboard, otherwise an error message is shown and no session is created. Logging out ends the session. This is provided by the Django authentication framework and is not a custom algorithm.

5.3 Architecture

Figure 5.10 shows the architecture of PackTrace: the web browser (client), the Django backend with its REST endpoints and WebSocket channel, the application database, and the external services — the device Geolocation source and the Google Maps Platform. No

separate API gateway, message broker or third-party notification service is part of this architecture.



Rider / User and Admin access the system through a standard web browser

The client layer consists of the login and registration pages, user dashboard, live ride map, ride controls, ride history, SOS alerts, and profile management pages. These interfaces run in the rider's web browser, where the browser's Geolocation API is used to obtain the rider's current GPS coordinates.

The backend layer is implemented using the Django web framework and contains modules for user authentication, rider profile management, ride group management, GPS location tracking, route deviation detection, ride history, SOS alert handling, and the Django Administration interface. Business logic is implemented in Django views and models.

The data layer consists of the application database accessed through Django's Object Relational Mapper (ORM). Application data, including users, rider profiles, ride groups, ride members, location logs, and SOS alerts, are stored and retrieved through Django models.

During a ride, the browser obtains GPS coordinates through the Geolocation API. Location updates are sent to the Django backend through WebSocket communication, with an HTTP location-update endpoint available as a fallback. The backend stores location records and distributes updates to connected group members. Road-route geometry is obtained from the OSRM routing service and displayed on Google Maps using the Maps JavaScript API.

5.4 Case Study

The following case study illustrates a typical PackTrace ride workflow.

Three riders, Rider A (the leader), Rider B and Rider C, plan a group ride. Rider A registers and logs in, creates a ride group and shares the group details. Riders B and C register, log in and join the group. When the ride starts, each rider's browser asks for permission to use the device location; after it is granted, the devices send their positions to the server, and all three markers appear on every rider's map. Rider B takes a wrong turn; the system detects the deviation from the planned route and the group is informed. Later, Rider C enters an area without network coverage and stops sending updates; the system marks Rider C as disconnected and keeps Rider C's last known location on the map, so that Riders A and B know where to wait. Rider C raises an SOS alert after the connection returns, and the other riders receive the alert. At the end of the ride, Rider A ends the ride, which is stored in the ride history, and each rider can view it later.

CHAPTER 6

IMPLEMENTATION DETAILS

6.1 Implementation Procedures

PackTrace was implemented as a web application using Python and the Django framework for the backend. The user interface was developed using HTML, CSS and JavaScript. The Google Maps JavaScript API provides map display, while the OSRM routing service supplies road-route geometry. The project was developed using Visual Studio Code as the code editor, with Draw.io and StarUML used for system diagrams.

The application follows the standard Django project structure. The main application is **tracker**, which contains the core business logic. The project includes Django models for **RiderProfile**, **RideGroup**, **RideMember**, **LocationLog**, and **SOSAlert**. URL routing is configured through Django's URL dispatcher, while request handling is implemented using Django views. The project also contains WebSocket routing and consumer modules to support real-time ride updates.

The application database is accessed through **Django's Object Relational Mapper (ORM)**. The configured database engine is **SQLite**, with application data stored in the db.sqlite3 database. In addition to the PackTrace models, Django's built-in authentication tables are used to manage users, groups, permissions, and sessions.

The authentication module allows riders to register and log in securely using Django's built-in authentication framework. User passwords are stored as hashed values using Django's password hashing mechanism, while user sessions are managed through Django's session framework. Standard Django CSRF middleware is enabled to protect form submissions and POST requests.

Ride management enables a rider to create a ride group and invite or allow other riders to join it. During an active ride, the browser obtains the rider's current position through the **HTML5 Geolocation API**. These GPS coordinates are transmitted to the Django backend, where they are stored in the **LocationLog** model and distributed to connected group members through **WebSocket communication**. The live dashboard updates rider positions on the Google Maps interface as new location updates arrive. If a rider becomes temporarily disconnected, the last recorded location remains visible until new location data are received.

The application continuously checks for route deviation using the **Haversine distance** between the rider's current location and the planned route. When the deviation exceeds the configured threshold (**500 metres**), the rider is marked as off-route and a route deviation notification is generated. Emergency situations are handled through the **SOS Alert** module, which broadcasts alerts to all members of the ride group.

When a ride is completed, ride information is stored in the database. Riders can later access the **Ride History** page to view previous rides, including ride details such as pack name, locations, planned distance, travelled distance, duration, rider role, ride status, and participating members.

Administration is provided through the **Django Administration site (/admin/)**. The administrator can manage **Users**, **Groups**, **Rider Profiles**, **Ride Groups**, **Ride Members**, **Location Logs**, and **SOS Alerts** using the built-in Django administration interface.

Table 6.1 Security measures

Aspect	Implemented / provided	Recommended for production
Authentication	Email and password login; masked password entry; Django authentication and sessions	Throttling of repeated login failures; password strength policy
Password protection	Password and confirmation at registration; hashed storage by Django	Enforce minimum strength; password reset by email
Authorization	Ride data limited to authenticated users; administrator access restricted	Check group membership on every request and WebSocket connection
CSRF protection	Django middleware and tokens for forms	Keep enabled; send the token with every state-changing AJAX request
Input validation	Form validation at registration and login; validation of location data	Validate coordinate ranges and update frequency
Location privacy	Explicit consent at registration; sharing limited to the ride group	Limit retention of location records
Transport security	Not implemented in the development environment.	HTTPS and WSS; the Geolocation API needs a secure context in modern browsers
API key protection	Google Maps API key is configured for development.	Restrict the Google Maps API key by domain and API; keep keys out of source code
Configuration	Development configuration uses Django settings	Disable debug mode; set allowed hosts; keep the secret key private; use a production database

6.2 Testing Methods and Results

Testing is intended to verify that the functions of PackTrace work as expected individually and together. Because the system depends on a device GPS, a network connection and several users at once, testing is carried out with multiple browser sessions or devices in addition to automated tests of the backend. The testing methods are functional testing of each feature, integration testing of the flow from login to ride history, validation testing of incorrect inputs, access-control testing of unauthorised access, and user acceptance testing with a group of users.

Testing Methods Used

- Functional testing to verify registration, login, logout, ride creation and joining.
- Real-time testing to verify GPS updates, WebSocket delivery and marker updates with several riders.

- Behaviour testing to verify disconnect detection, last known location and route deviation.
- Alert testing to verify the SOS alert and the notification received by other riders.
- Data testing to verify that ride history is stored and displayed.
- Security testing to verify access control, administrator access and CSRF behaviour.

User Acceptance Test Cases

Table 6.2 lists the user acceptance test cases. The expected result of each case is defined from the intended behaviour of the system.

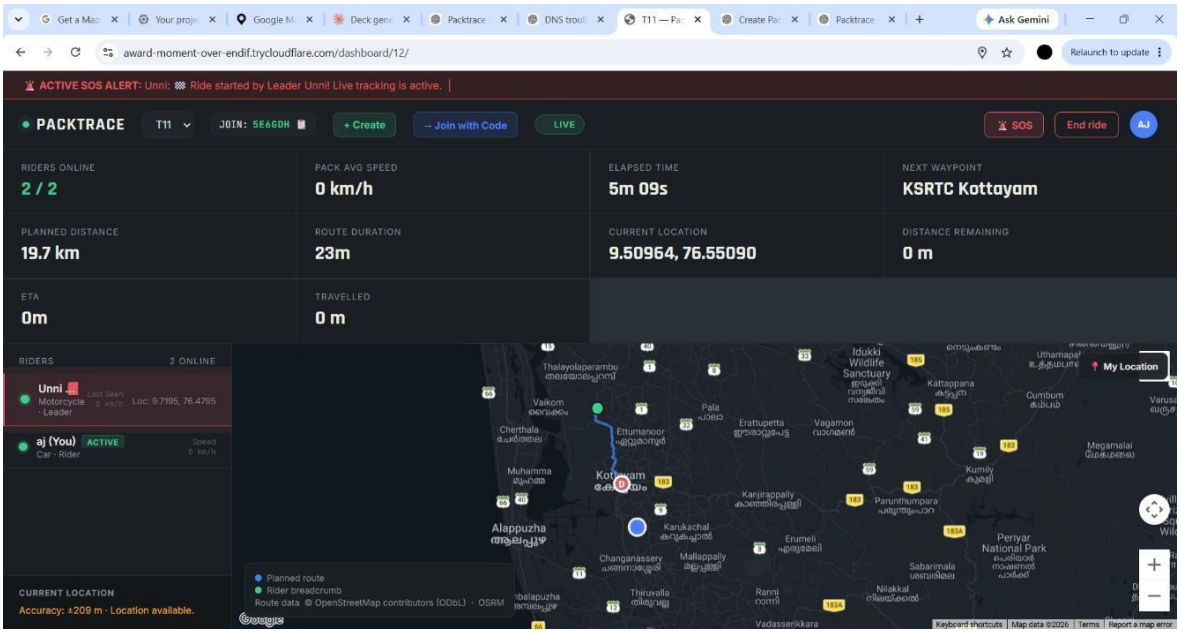
Table 6.2 User acceptance test cases

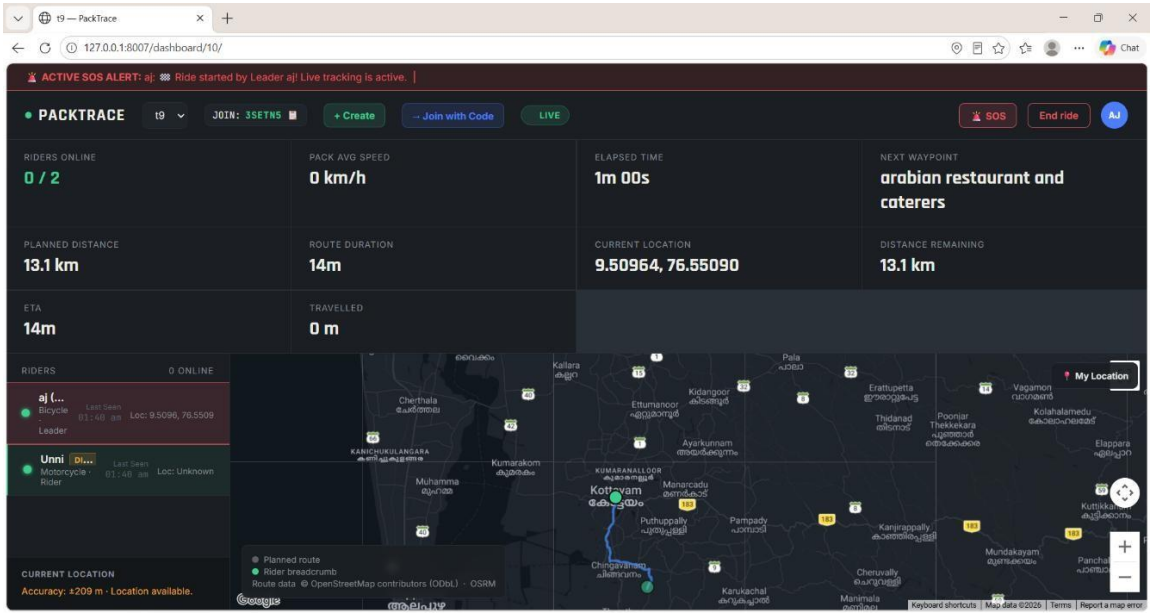
ID	Feature	Input / Action	Expected Result	Actual Result	Status
UAT01	User registration	Submit valid name, email, phone, ride type, matching passwords, consent	Account created; user can sign in	executed	pass
UAT02	User login	Valid email and password	Session created; dashboard shown	executed	pass
UAT03	Invalid login	Wrong password	Error message; no session	executed	pass
UAT04	Logout	Click logout	Session ended; login page shown	executed	pass
UAT05	Ride creation	Create a ride group	Group created; creator is leader	executed	pass
UAT06	Ride joining	Join with the valid group mechanism	Rider added to the group	executed	pass
UAT07	Multiple riders joining	Three or more users join the same ride	All appear as members of the ride	executed	pass
UAT08	GPS permission	Deny, then allow, location permission	Message when denied; sharing starts when allowed	executed	pass
UAT09	Live GPS update	Move the device	New location is sent and stored	executed	pass
UAT10	Multiple rider markers	Several riders share location	A marker for each rider on the map	executed	pass
UAT11	WebSocket update	Rider A moves	Rider B's map updates without reload	executed	pass
UAT12	Disconnect detection	Stop updates from one rider	Rider shown as disconnected	executed	pass
UAT13	Last known location	Rider disconnected	Last known location stays on the map	executed	pass
UAT14	Route display	Open an active ride	Planned route and rider path displayed	executed	pass
UAT15	Route deviation	Move away from the route	Deviation detected and reported	executed	pass
UAT16	SOS alert	Press SOS	Alert raised and stored	executed	pass
UAT17	Notification	SOS raised by Rider A	Other riders receive the alert	executed	pass

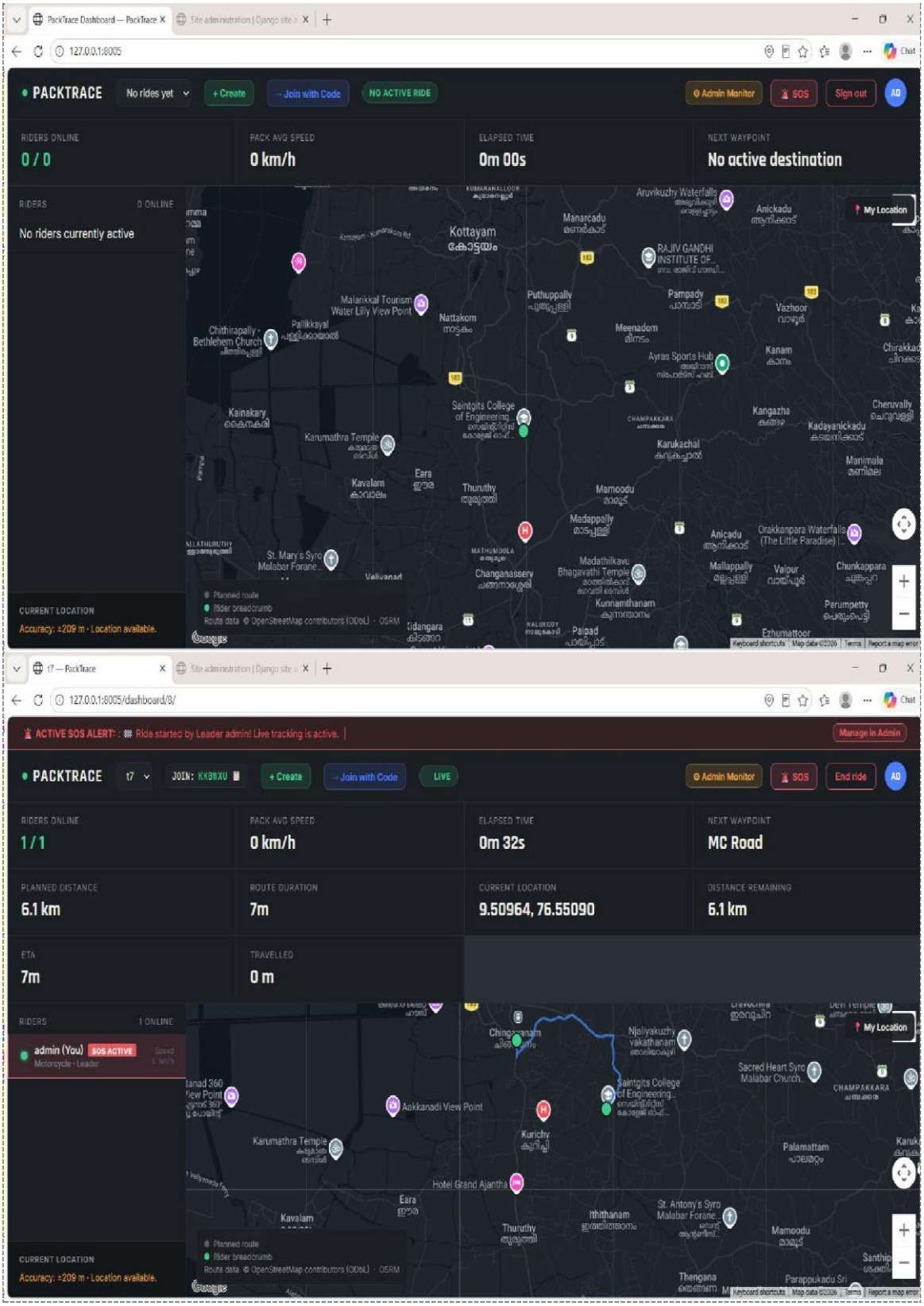
UAT18	End ride	End the ride	Sharing stops; ride stored	executed	pass
UAT19	Ride history	Open ride history	Completed ride listed with stored details	executed	pass
UAT20	Admin access	Log in as administrator	Administration interface available	executed	pass
UAT21	Unauthorized access protection	Open protected pages without login; open admin as normal user	Redirected to login / access denied	executed	pass
UAT22	CSRF / security behaviour	Submit a POST without a CSRF token	Request rejected (403)	executed	pass

Test Results

The test cases in Table 6.2 are marked as executed and passed. Figure 6.1 provides screenshots as evidence of user acceptance testing.


award-moment-over-endif.trycloudflare.com/dashboard/12/
ACTIVE SOS ALERT: Unni: Ride started by Leader Unni! Live tracking is active.
PACKTRACE
T11
JOIN: 5E6GDH
+ Create
→ Join with Code
LIVE
SOS
End ride
AJ
RIDERS ONLINE
2 / 2
PACK AVG SPEED
0 km/h
ELAPSED TIME
5m 09s
NEXT WAYPOINT
KSRTC Kottayam
PLANNED DISTANCE
19.7 km
ROUTE DURATION
23m
CURRENT LOCATION
9.50964, 76.55090
DISTANCE REMAINING
0 m
ETA
0m
TRAVELLED
0 m
RIDERS
2 ONLINE
Unni
Motorcycle · Leader
Loc: 9.7195, 76.4795
aj (You)
ACTIVE
Car · Rider
CURRENT LOCATION
Accuracy: ±209 m · Location available.
My Location
Planned route
Rider breadcrumb
Route data © OpenStreetMap contributors (ODbL) · OSRM


127.0.0.1:8007/dashboard/10/
ACTIVE SOS ALERT: aj: Ride started by Leader aj! Live tracking is active.
PACKTRACE
t9
JOIN: 3SETN5
+ Create
→ Join with Code
LIVE
SOS
End ride
AJ
RIDERS ONLINE
0 / 2
PACK AVG SPEED
0 km/h
ELAPSED TIME
1m 00s
NEXT WAYPOINT
arabian restaurant and caterers
PLANNED DISTANCE
13.1 km
ROUTE DURATION
14m
CURRENT LOCATION
9.50964, 76.55090
DISTANCE REMAINING
13.1 km
ETA
14m
TRAVELLED
0 m
RIDERS
0 ONLINE
aj (...
Bicycle · Leader
Loc: 9.5096, 76.5509
Unni
Motorcycle · Rider
Loc: Unknown
CURRENT LOCATION
Accuracy: ±209 m · Location available.
My Location
Planned route
Rider breadcrumb
Route data © OpenStreetMap contributors (ODbL) · OSRM


PackTrace Dashboard — PackTrace
127.0.0.1:8005
PACKTRACE
No rides yet
+ Create
Join with Code
NO ACTIVE RIDE
Admin Monitor
SOS
Sign out
AD
RIDERS ONLINE
0 / 0
PACK AVG SPEED
0 km/h
ELAPSED TIME
0m 00s
NEXT WAYPOINT
No active destination
RIDERS
0 ONLINE
No riders currently active
CURRENT LOCATION
Accuracy: ±209 m · Location available.
My Location
Kottayam
Aruvikuzhy Waterfalls
Anickadu
Manarcadu
RAJIV GANDHI INSTITUTE OF...
Malarikkal Tourism Water Lilly View Point
Nattakom
Puthuppally
Pampady
Vazhoor
Meenadom
Ayras Sports Hub
Kanam
Chithirapally - Bethlehem Church
Pallikkayal
Saintgits College of Engineering...
Kainakary
Karumathra Temple
Kangazha
Cheruvally
Kadayanickadu
Karukachal
Manimala
Kavalam
Eara
Thuruthy
Mamoodu
Madappally
Aniçadu
Orakkanpara Waterfalls (The Little Paradise)
St. Mary's Syro Malabar Forane...
Changanassery
Madathilkavu Bhagavathi Temple
Mallappally
Vaipur
Chunkappara
Kunnamthanam
Perumpetty
Ezhumattoor
Valiyanad
Paipad
Planned route
Rider breadcrumb
Route data © OpenStreetMap contributors (ODbL) · OSRM
Google
t7 — PackTrace
127.0.0.1:8005/dashboard/8/
ACTIVE SOS ALERT: Ride started by Leader admin! Live tracking is active.
Manage in Admin
PACKTRACE
t7
JOIN: KKBWXU
+ Create
Join with Code
LIVE
Admin Monitor
SOS
End ride
AD
RIDERS ONLINE
1 / 1
PACK AVG SPEED
0 km/h
ELAPSED TIME
0m 32s
NEXT WAYPOINT
MC Road
PLANNED DISTANCE
6.1 km
ROUTE DURATION
7m
CURRENT LOCATION
9.50964, 76.55090
DISTANCE REMAINING
6.1 km
ETA
7m
TRAVELLED
0 m
RIDERS
1 ONLINE
admin (You)
SOS ACTIVE
Motorcycle · Leader
CURRENT LOCATION
Accuracy: ±209 m · Location available.
My Location
Chingavanam
Njaliyakuzhy
Vakathanam
Sacred Heart Syro Malabar Church...
CHAMPAKKARA
Saintgits College of Engineering...
Aakkanadi View Point
Kurichy
Karumathra Temple
Palamattam
Hotel Grand Ajantha
Eara
Kavalam
Thuruthy
St. Antony's Syro Malabar Forane...
Mamoodu
Thengana
Parappukadu Sri
Planned route
Rider breadcrumb
Route data © OpenStreetMap contributors (ODbL) · OSRM
Google

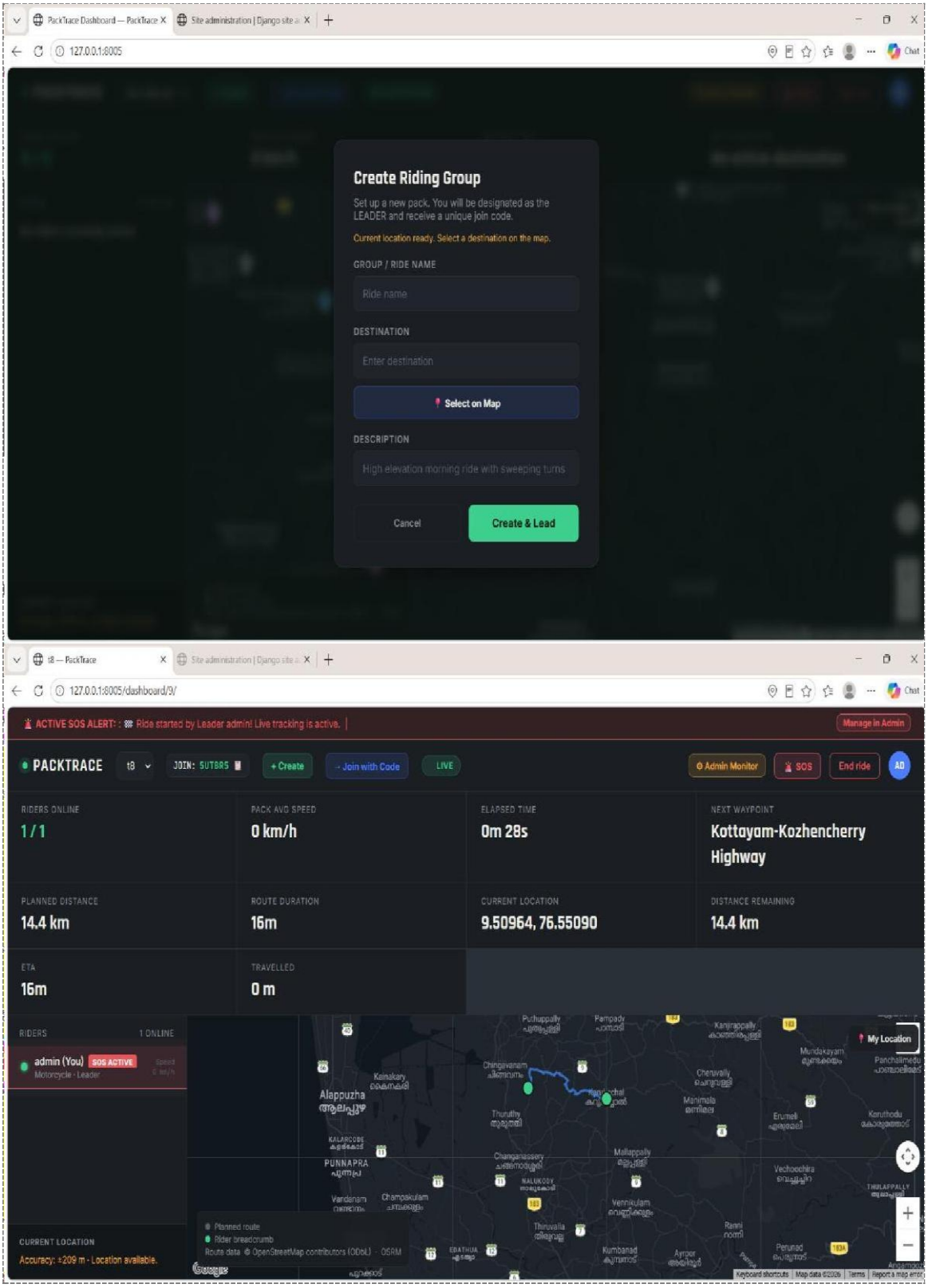


Figure 6.1 Evidence of user acceptance testing

CHAPTER 7

ADVANTAGES AND DISADVANTAGES

7.1 Advantages

- **Centralised group tracking.** All riders appear on one map instead of in separate chats.
- **Real-time location awareness.** Members see the latest known position of every rider, delivered over WebSocket communication.
- **Dedicated ride dashboard.** The map, rider list and ride information are presented together.
- **Group ride management.** Rides are created and joined by riders, with a leader and members.
- **Safety functions.** Disconnected riders are marked and their last known location is retained; route deviation is detected; an SOS alert notifies the group.
- **Ride history.** Completed rides can be reviewed later.
- **Web accessibility.** The application runs in a browser and needs no installation.
- **Open technology base.** Python, Django and Django Channels are free and open source.

7.2 Disadvantages

- GPS accuracy depends on the device and the surroundings.
- Tracking depends on network connectivity; updates are delayed or lost where coverage is poor.
- Browsers generally limit location access when the page is in the background or the screen is off, which affects continuous tracking in a web application.
- The map service depends on API configuration, an API key and usage limits.
- Continuous tracking increases battery consumption.
- PackTrace is a web application; it is not a native mobile application.

CHAPTER 8

RESULT AND DISCUSSION

8.1 Analysis

This chapter discusses the extent to which **PackTrace** achieved the objectives presented in Chapter 1. The discussion focuses on the implemented system and its major features.

The implemented interface provides a **user registration page**, which records the rider's consent for location sharing, a **login page**, and a **dashboard** that integrates ride controls, ride status, ride statistics, a rider list, and a Google Maps-based interface displaying rider markers, the planned route, and the rider's travelled breadcrumb path. These features satisfy the objective of providing a centralized dashboard for monitoring group rides.

The application successfully implements **ride group creation and joining**, **real-time GPS location sharing** using **WebSocket communication**, **live rider tracking** on a **Google Maps Platform** interface, **detection of disconnected riders** while retaining their last known location, **route deviation detection** using the **Haversine distance algorithm**, an **SOS alert system**, **ride history**, and **administration through the Django Administration site**. These features fulfil the objectives of real-time location sharing, group ride management, rider safety, emergency communication, and post-ride record keeping.

Route deviation is detected by comparing the rider's current GPS coordinates with the planned route using the **Haversine formula** and a configurable deviation threshold. Ride information is stored when a ride ends and is available through the **Ride History** module. Administrative functions allow authorized administrators to manage users, ride groups, ride members, location logs, and SOS alerts through the Django Admin interface.

Although the system demonstrates the required functionality, its performance has not been formally evaluated. Performance can be influenced by factors such as GPS update frequency, WebSocket latency, network quality, the number of riders participating in a ride, database write frequency for location updates, browser rendering performance, and the responsiveness of Google Maps services. The use of **WebSocket communication** enables continuous real-time updates without relying on repeated HTTP polling.

Table 8.1 summarises the status of each objective.

Objective	Status in this report
Real-time location sharing	Implemented
Create and join ride groups	Implemented
Common map and centralised dashboard	Implemented
Track movement and route	Implemented
Awareness of separated riders	Implemented
Route deviation detection	Implemented
SOS emergency alert	Implemented
Ride history	Implemented
Secure authentication and administration	Implemented

Table 8.1 Status of the objectives

CHAPTER 9

CONCLUSION

9.1 Summary

PackTrace was developed to address the practical challenges of monitoring riders during group rides. Traditional messaging applications allow users to share their individual locations but do not provide a centralized platform for managing and tracking an entire riding group in real time. PackTrace overcomes this limitation by providing a web-based application that enables riders to create or join ride groups and view all participating riders on a common map.

The system integrates **browser-based GPS location tracking**, **real-time location sharing using WebSocket communication**, **Google Maps Platform** for route visualization, **ride group management**, **disconnected rider detection**, **route deviation detection using the Haversine algorithm**, an **SOS emergency alert system**, **ride history**, and **administration through the Django Administration site**. The application is built using **Python**, **Django**, **Django Channels**, and **SQLite**, with **Django ORM** handling database operations. The modular architecture allows the system to be extended with additional features in the future.

The project successfully demonstrates the implementation of a real-time group ride tracking system that improves rider coordination and safety. Riders can monitor each other's locations, receive emergency alerts, review completed rides, and manage ride groups through an intuitive web interface. The implemented features meet the objectives defined at the beginning of the project.

Although the application performs the required functions, further improvements can be made for deployment in real-world environments. Future enhancements may include performance optimization for larger rider groups, deployment over **HTTPS/WSS**, push notifications, mobile application support, stronger security policies, integration with cloud databases, and advanced analytics for ride statistics.

Overall, **PackTrace** provides a practical and scalable foundation for real-time group ride tracking and demonstrates the effective use of modern web technologies to improve rider coordination, navigation, and safety.

9.2 Limitations

- GPS accuracy depends on the device and the surroundings.
- Real-time tracking depends on network connectivity.
- Browser and device permissions are required, and background location access is restricted in web browsers.

- Google Maps Platform functionality depends on API configuration, keys and usage limits.
- Continuous location tracking increases battery consumption.

9.3 Future Scope

The following enhancements are future scope and are not features of the present system.

- Native mobile application for Android and iOS, enabling reliable background location sharing.
- Offline tracking and SMS fallback for areas without data coverage.
- Geofencing, low-battery alerts and no-movement detection.
- Improved emergency communication, including voice or audio emergency recording and push, SMS or email notifications.
- Automatic accident detection.
- Advanced ride analytics and sharing of ride summaries.
- Cloud deployment and support for larger numbers of concurrent riders.

9.4 Applications

- Motorcycle group rides and touring.
- Road trips of friends and families.
- Organised riding communities and clubs.
- Event-based group travel such as rallies.
- Cycling groups and mixed groups of riders.
- Safety-oriented group monitoring on ghat and highway routes.

CHAPTER 10

APPENDICES

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Sign in — PackTrace</title>
<link rel="stylesheet" href="{% static 'css/packtrace.css' %}">
</head>
<body>
<div class="auth-shell">

  <!-- Hero / signature panel -->
  <section class="auth-hero">
    <div class="hero-top">
      <div class="wordmark"><span class="pulse-dot"></span>PACKTRACE</div>
      <p class="hero-tagline">Live GPS for your whole riding group — one map,
every rider, real time.</p>
    </div>

    <div class="hero-trace-wrap">
      <svg viewBox="0 0 460 500" fill="none" xmlns="http://www.w3.org/2000/svg">
        <path class="trace-path" d="M40,460 C110,430 70,360 150,325 C230,290
190,220 270,185 C330,158 300,95 380,55"/>
        <circle class="trace-waypoint" style="animation-delay:.6s" cx="40" cy="460"
r="5"/>
        <circle class="trace-waypoint" style="animation-delay:1.4s" cx="150"
cy="325" r="5"/>
        <circle class="trace-waypoint" style="animation-delay:2.2s" cx="270"
cy="185" r="5"/>
```

```
      <circle class="trace-head" style="animation-delay:3.1s" cx="380" cy="55"
r="6"/>
    </svg>
   </div>


 </section>


 <!-- Form panel -->
 <section class="auth-form-panel">
  <div class="auth-card">
   <h1>Sign in to your pack</h1>
   <p class="sub">Track your riders live, or get tracked by them.</p>


   {% if error_message %}
   <div class="server-alert danger">
    <span>⚠</span>
    <span>{{ error_message }}</span>
   </div>
   {% endif %}


   <form id="login-form" method="POST" action="{% url 'login' %}"
novalidate>
     {% csrf_token %}
     <div class="field" id="field-email">
      <label for="email">Email</label>
      <input type="email" id="email" name="email" value="{{ email_val }}"
placeholder="you@example.com" autocomplete="email" required>
      <div class="field-error">Enter a valid email address.</div>
     </div>
```

CHAPTER 11

REFERENCES

[1] E. A. Aldhahri, A. A. Almazroi, M. H. Alkinani, M. Alqarni, E. A. Alghamdi, and N. Ayub, "GNN-RMNet: Leveraging graph neural networks and GPS analytics for driver behavior and route optimization in logistics," *PLOS ONE*, vol. 20, no. 8, Art. no. e0328899, Aug. 2025, doi: 10.1371/journal.pone.0328899.

[2] I. Moumen, N. Rafalia, J. Abouchabaka, and M. Aoufi, "Real-time GPS tracking system for IoT-enabled connected vehicles," *E3S Web Conf.*, vol. 412, Art. no. 01095, 2023, doi: 10.1051/e3sconf/202341201095.

[3] P. Boronat, M. Pérez-Francisco, C. T. Calafate, and J.-C. Cano, "Towards a sustainable city for cyclists: Promoting safety through a mobile sensing application," *Sensors*, vol. 21, no. 6, Art. no. 2116, Mar. 2021, doi: 10.3390/s21062116.

[4] V. Pimentel and B. G. Nickerson, "Communicating and displaying real-time data with WebSocket," *IEEE Internet Comput.*, vol. 16, no. 4, pp. 45–53, Jul./Aug. 2012, doi: 10.1109/MIC.2012.64.

[5] P. A. Zandbergen and S. J. Barbeau, "Positional accuracy of assisted GPS data from high-sensitivity GPS-enabled mobile phones," *J. Navigation*, vol. 64, no. 3, pp. 381–399, Jul. 2011, doi: 10.1017/S0373463311000051.

[6] J. White, C. Thompson, H. Turner, B. Dougherty, and D. C. Schmidt, "WreckWatch: Automatic traffic accident detection and notification with smartphones," *Mobile Netw. Appl.*, vol. 16, no. 3, pp. 285–303, Jun. 2011, doi: 10.1007/s11036-011-0304-8.

[7] P. A. Zandbergen, "Accuracy of iPhone locations: A comparison of assisted GPS, WiFi and cellular positioning," *Trans. GIS*, vol. 13, no. s1, pp. 5–25, 2009, doi: 10.1111/j.1467-9671.2009.01152.x.

[8] S. Consolvo, I. E. Smith, T. Matthews, A. LaMarca, J. Tabert, and P. Powledge, "Location disclosure to social relations: Why, when, & what people want to share," in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI '05)*, Portland, OR, USA, 2005, pp. 81–90, doi: 10.1145/1054972.1054985.

[9] A. R. Beresford and F. Stajano, "Location privacy in pervasive computing," *IEEE Pervasive Comput.*, vol. 2, no. 1, pp. 46–55, Jan.–Mar. 2003, doi: 10.1109/MPRV.2003.1186725.

[10] R. W. Sinnott, "Virtues of the Haversine," *Sky & Telescope*, vol. 68, no. 2, p. 159, Aug. 1984.

[11] A. Popescu, Ed., "Geolocation API," W3C Recommendation, Sep. 1, 2022. [Online]. Available: https://www.w3.org/TR/2022/REC-geolocation-20220901/

[12] Google, "Google Maps Platform documentation: Maps JavaScript API." [Online]. Available: https://developers.google.com/maps/documentation/javascript

[13] Python Software Foundation, "Python documentation." [Online]. Available: https://docs.python.org/

[14] Django Software Foundation, "Django documentation." [Online]. Available: https://docs.djangoproject.com/

[15] Django Software Foundation, "Channels documentation." [Online]. Available: https://channels.readthedocs.io/

[16] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF, RFC 6455, Dec. 2011. [Online]. Available: https://www.rfc-editor.org/info/rfc6455

[17] OWASP Foundation, "OWASP Top Ten." [Online]. Available: https://owasp.org/www-project-top-ten/

ALAN JOHNY

PackTrace_Mini_Project_Report_New

mini project report

Document Details

Submission ID

trn:oid:::10159:154180459

Submission Date

Oct 10, 2026, 12:07 AM GMT+5:30

Download Date

Oct 10, 2026, 12:12 AM GMT+5:30

File Name

PackTrace_Mini_Project_Report_New.pdf

File Size

2.9 MB

67 Pages

15,224 Words

84,991 Characters

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 15 words)

Match Groups

31 Not Cited or Quoted 9%
Matches with neither in-text citation nor quotation marks

2 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
3% Publications
8% Submitted works (Student Papers)